Code Notes on the HNSW Implementation in this Repository

Contents

1 Where the Main Implementation Lives **5**

2 CPU or GPU **6**

3 Public Construction and Query Parameters **7**

4 Distance Spaces and Input Vector Format **8**

5 Internal Node Identifiers and Labels **9**

6 Level Assignment and Entry Point **10**

7 Memory Layout of Node Data **11**

8 How Link Lists Are Stored **12**

9 Graph Construction Path **13**

10 Neighbor Selection and Reciprocal Updates **14**

11 Detailed Control Flow of addPoint **15**

11.1 Clarifications About Nodes, Layers, Edges, Memory Access, and Deletion 15
11.2 Public Wrapper: Replacement Policy 16
11.3 Replacement Path for Deleted Elements 17
11.4 Internal Overload: New Insert or Existing-Label Update 17
11.5 Level Selection and Locking 18
11.6 Writing the Node's Stored Data 19
11.7 First-Node Case 19
11.8 Greedy Descent from Upper Layers 19
11.9 Construction Search and Mutual Connection 20
11.10 Entry Point Update 20
11.11 Clarification: Difference Between addPoint and updatePoint 21
11.12 Short Logical Summary 22

12 Query Path **23**

13 Original Data Handling **24**

14 Index Save File and Load File Format **25**

15 Deletion and Replacement **26**

16 Locking Mechanism During Construction and Deletion **27**
16.1 Why Locks Are Required During HNSW Graph Construction 27
16.2 Meaning of lock_deleted_elements 27
16.3 When a Node Is Marked Deleted 28
16.4 When a Node Is Unmarked Deleted 28
16.5 Relationship Between Deletion, Un-Deletion, and Replacement 29

17 Threading and Locks **30**

18 Mechanism of the C++ Examples **31**
18.1 Basic Search: example_search.cpp 31
18.2 Filtered Search: example_filter.cpp 31
18.3 Multithreaded Search: example_mt_search.cpp 32
18.4 Multithreaded Filtered Search: example_mt_filter.cpp 32
18.5 Epsilon Search: example_epsilon_search.cpp 33
18.6 Multivector Search: example_multivector_search.cpp 34

19 Distance-Space Implementation: space_ip.h and space_l2.h **35**
19.1 Mechanism of space_ip.h 35
19.2 Mechanism of space_l2.h 36
19.3 How the Distance Space Is Used by HNSW 37
19.4 Controlling SSE, AVX, and AVX-512 38
19.4.1 Summary for the Current Machine 38

20 Constructing HNSW from the Compressed CPRQ Package Without Full Decompression **40**
20.1 What the Package Actually Stores 40
20.2 Reconstruction Model and Exact L2 Distance 40
20.3 Residual Random Access Is a Required Index 41
20.4 Why Stock L2Space Cannot Store the Compressed Rows 42
20.5 Option 1: Stream One Reconstructed Row into Ordinary HNSW 42
20.6 Option 2: Store Row Descriptors and Decode Distances on Demand 43
20.7 Option 3: Build HNSW on Compact Surrogate Codes and Rerank 44
20.8 Option 4: Block-Level Graph or Two-Level Index 45
20.9 Exact, Fast, and Corrected Compressed Distances 46
20.10Can SIMD Be Used? 46
20.11Why a Per-Distance GPU Call Is Not Appropriate 47
20.12Caching and Multithreading Considerations 48
20.13Persistence and Validation Requirements 48
20.14Recommended Development Sequence 49
20.15Overall Recommendation 49

21 Proposed HNSW Nodes with Row-Wise Signed Residual Bins, Alpha, and Beta **51**
21.1 Restatement of the Compressed Model 51
21.2 From Dimension-Major Blocks to Row-Major Node Codes 52
21.3 Selecting Signed 8-Bit or Signed 16-Bit Storage 52
21.4 Global State and Per-Node State 53

21.5 Beta Computed by the Existing CUDA Method . . . 53
21.6 Database-to-Database Distance Derivation . . . 54
21.7 Relation to the Current Corrected CUDA Distance . . . 54
21.8 Recommended Exact QR Reparameterization . . . 55
21.9 Query-to-Node Distance . . . 55
21.10 Required Change: Keep Every Query in Full Precision . . . 56
21.11 Minimal-Change Integration Without Modifying hnswalg.h . . . 56
21.12 Full-Precision Query Preparation and Distance . . . 57
21.13 Lifetime and Multithreaded Query Rules . . . 58
21.14 Full-Precision Query Pseudocode with Minimal Changes . . . 58
21.15 SIMD with a Full-Precision Query . . . 59
21.16 Per-Query Signed-Bin Distance Lookup Tables . . . 59
21.16.1 Alignment and Mixed FP32/Integer Types . . . 59
21.16.2 Full Per-Query Table Size . . . 60
21.16.3 Multiple Dimension-Wise Positive and Negative Tables . . . 60
21.16.4 Why the Access Pattern Can Still Be Slow . . . 63
21.16.5 Pros of the Lookup-Table Design . . . 63
21.16.6 Cons of the Lookup-Table Design . . . 64
21.16.7 Lower-Memory Algebraic Alternative . . . 64
21.16.8 Tiled and Hybrid Lookup Alternatives . . . 64
21.16.9 Recommendation . . . 65
21.17 Optional Cleaner API Change . . . 65
21.18 Compatibility with the hnswlib Distance Interface . . . 65
21.19 Proposed Data-Preparation Algorithm . . . 66
21.20 Proposed HNSW Construction Algorithm . . . 66
21.21 Proposed HNSW Query Algorithm . . . 67
21.22 SIMD Implementation Opportunities . . . 67
21.23 Memory Consequences for D=3584 and R=64 . . . 68
21.24 KMeans and Cluster-Centered Models . . . 68
21.25 Feasibility Verdict . . . 69
21.26 Required Correctness Checks Before Code Generation . . . 70
21.27 Recommended Design Decision . . . 70

22 Implemented CPU CPRQ HNSW Construction and Full-Precision Search 71
22.1 Package Input and Outer Zstd Handling . . . 71
22.2 Raw Full-Package Parsing . . . 71
22.3 Actual Block-Mask Residual Layout . . . 72
22.4 Signed Int8 Conversion and Clipping . . . 73
22.5 QR Transformation of Alpha and Beta . . . 73
22.6 Fixed HNSW Node Payload . . . 74
22.7 Exact Node-to-Node Construction Distance . . . 74
22.8 Final HNSW Construction Algorithm . . . 74
22.8.1 Summary: Why the CPU Applies QR but the GPU Does Not . . . 75
22.9 Full-Precision Query Preparation . . . 76
22.10 Exact Query-to-Node Distance Without a Cache Table . . . 77
22.11 Final SearchKNN Algorithm . . . 77
22.12 Measured Validation Results . . . 78
22.13 Build and Run Location . . . 78

23 Summary **79**

1 Where the Main Implementation Lives

The HNSW implementation in this checkout is mostly header-only C++. The central class is `hnswlib::HierarchicalNSW<dist_t>` in `hnswlib/hnswalg.h`. The general interfaces, distance-function types, binary read/write helpers, and public algorithm interface are in `hnswlib/hnswlib.h`. The distance spaces are implemented in `hnswlib/space_l2.h` and `hnswlib/space_ip.h`. The Python API is a pybind11 wrapper in `python_bindings/bindings.cpp`.

There is also a separate brute-force baseline in `hnswlib/bruteforce.h`, exposed to Python as `hnswlib.BFIndex`. That file is useful for comparison, but it is not the HNSW graph implementation.

2 CPU or GPU

This implementation is CPU based. The HNSW construction and query code uses ordinary C++ memory allocation, pointer arithmetic, mutexes, priority queues, and CPU distance functions. No CUDA kernels, GPU memory allocations, or GPU launch paths are present in the HNSW code.

The code does contain CPU SIMD optimizations. In `hnswlib.h`, compile-time macros enable `USE_SSE`, `USE_AVX`, and `USE_AVX512` when the compiler and target CPU support them. The L2 and inner-product distance spaces then select vectorized CPU functions, for example `L2SqrSIMD16ExtAVX` or `InnerProductDistanceSIMD16ExtAVX512`. The graph traversal also uses CPU prefetch intrinsics such as `_mm_prefetch` under `USE_SSE`. These are CPU cache/SIMD optimizations, not GPU execution.

3 Public Construction and Query Parameters

The Python binding exposes the HNSW parameters through `Index.init_index`:

```
init_index(max_elements, M=16, ef_construction=200,
          random_seed=100, allow_replace_deleted=False)
```

The same defaults appear in the C++ constructor of `HierarchicalNSW`:

```
HierarchicalNSW(SpaceInterface<dist_t> *s,
               size_t max_elements,
               size_t M = 16,
               size_t ef_construction = 200,
               size_t random_seed = 100,
               bool allow_replace_deleted = false)
```

The important fields are:

- `max_elements_`: fixed capacity of the index allocation. The index can be resized later with `resizeIndex`.
- `cur_element_count`: current number of internal elements. Internal node ids are dense integers from 0 to `cur_element_count - 1`.
- `M_`: user requested graph degree parameter. If `M > 10000` it is capped to 10000.
- `maxM_`: maximum number of neighbors per node on upper layers. It is set to `M_`.
- `maxM0_`: maximum number of neighbors per node on layer 0. It is set to `2 * M_`.
- `ef_construction_`: candidate-list width during insertion. The constructor stores `max(ef_construction , M_)`, so construction search is never narrower than `M_`.
- `ef_`: query-time search width. It defaults to 10 and can be changed from Python with `set_ef` or the writable `ef` property.
- `random_seed`: seeds the level generator and the update probability generator.
- `allow_replace_deleted_`: enables reusing slots that were marked deleted.

The search call exposes:

```
knn_query(data, k=1, num_threads=-1, filter=None)
```

The actual base-layer search width is `max(ef_, k)`. Thus `ef` should be at least `k`, and larger values usually improve recall at the cost of more distance computations.

4 Distance Spaces and Input Vector Format

The public Python constructor is:

```
hnswlib.Index(space, dim)
```

The supported HNSW spaces are:

- `l2`: squared Euclidean distance, implemented by `L2Space` in `space_l2.h`.
- `ip`: inner-product distance `1.0 - dot(a,b)`, implemented by `InnerProductSpace` in `space_ip.h`.
- `cosine`: uses the inner-product space internally, but the Python binding normalizes vectors before insertion and query.

The HNSW class stores a generic distance-function pointer:

```
DISTFUNC<dist_t> fstdistfunc_;
void *dist_func_param_;
size_t data_size_;
```

For the normal Python bindings, `dist_t` is `float` and the stored vectors are contiguous `float32` vectors of length `dim`. The binding converts input arrays with `py::array_t<dist_t, py::array::c_style` py::array::forcecast¿—, checks the dimensionality, optionally normalizes cosine vectors, and passes a row pointer into `HierarchicalNSW::addPoint`.

After insertion, the index owns its own copy of the vector bytes. It does not keep a reference to the original NumPy array or original input buffer.

5 Internal Node Identifiers and Labels

The implementation separates external labels from internal node ids.

- External labels have type `labeltype`, defined as `size_t` in `hnswlib.h`.
- Internal ids have type `tableint`, defined as `unsigned int` in `hnswalg.h`.
- `label_lookup_` maps external labels to internal ids.
- `getExternalLabel(internal_id)` reads the stored external label from the node's memory block.

If the Python user does not provide ids in `add_items`, the binding uses sequential labels. If the user provides ids, those become the external labels. Search returns labels, not internal ids.

6 Level Assignment and Entry Point

Each node receives a random maximum layer from:

```
int getRandomLevel(double reverse_size) {
    double r = -log(U(0,1)) * reverse_size;
    return (int) r;
}
```

The constructor sets:

```
mult_ = 1 / log(1.0 * M_);
revSize_ = 1.0 / mult_;
```

The sampled level is stored in `element_levels_[internal_id]`. The index also keeps:

- `maxlevel_`: highest level currently present in the graph.
- `enterpoint_node_`: internal id used as the starting point for top-down graph traversal.

The first inserted node becomes the entry point. If a later inserted node has a higher level than the previous maximum, it becomes the new entry point and `maxlevel_` is updated.

7 Memory Layout of Node Data

The most important storage decision is that layer 0 links, vector data, and external label are stored together in one large contiguous allocation:

```
char *data_level0_memory_;
```

For each node, the byte layout is:

```
[ level-0 link list ][ raw vector bytes ][ external label ]
```

The constructor computes:

```
size_links_level0_ = maxM0_ * sizeof(tableint)
                   + sizeof(linklistsizeint);
size_data_per_element_ = size_links_level0_
                       + data_size_
                       + sizeof(labeltype);
offsetLevel0_ = 0;
offsetData_ = size_links_level0_;
label_offset_ = size_links_level0_ + data_size_;
```

Therefore:

- `get_linklist0(id)` returns the beginning of the layer-0 neighbor list for node `id`.
- `getDataByInternalId(id)` returns the raw vector bytes for node `id`.
- `getExternalLabeLp(id)` returns the label address for node `id`.

Upper-layer links are stored separately:

```
char **linkLists_;
size_links_per_element_ = maxM_ * sizeof(tableint)
                        + sizeof(linklistsizeint);
```

For a node with level greater than zero, `linkLists_[id]` points to a block containing that node's link lists for levels 1 through `element_levels_[id]`. The helper `get_linklist(id, level)` indexes into this block with:

```
linkLists_[id] + (level - 1) * size_links_per_element_
```

8 How Link Lists Are Stored

Each link list starts with a small count field, followed by an array of neighbor internal ids:

```
[ link-list count ][ neighbor id 0 ][ neighbor id 1 ] ...
```

The neighbor ids are `tableint` values. The count is manipulated through:

```
getListCount(linklistsizeint *ptr)
setListCount(linklistsizeint *ptr, unsigned short int size)
```

Layer 0 can hold up to `maxM0_ = 2M` neighbors per node. Upper layers can hold up to `maxM_ = M` neighbors per node.

The graph is therefore an adjacency-list graph, but it is stored in compact manual memory blocks rather than in `std::vector<std::vector<int>>`. During traversal, the code obtains a pointer to the link list, reads the count, and then scans the neighbor id array.

9 Graph Construction Path

Insertion starts in `HierarchicalNSW::addPoint(data_point, label, level)`.

The insertion path is:

1. Check whether the label already exists in `label_lookup_`. If it exists, the node is updated instead of creating a duplicate.
2. Assign a new dense internal id from `cur_element_count`.
3. Sample the node level with `getRandomLevel(mult_)` unless a positive level is explicitly supplied.
4. Zero the node's level-0 memory block.
5. Copy the external label into the label region.
6. Copy the input vector bytes into the data region.
7. Allocate upper-level link memory if the sampled level is greater than zero.
8. From the current entry point, greedily descend upper levels until the insertion level is reached.
9. For each level from `min(curlevel, maxlevelcopy)` down to zero, run `searchBaseLayer` with `ef_construction_` width to collect candidate neighbors.
10. Use `mutuallyConnectNewElement` to select neighbors, write the new node's outgoing links, and update reciprocal links in neighboring nodes.

The construction search uses two priority queues. One queue stores candidates to explore, and another stores the best candidates found so far. The visited state is kept in a reusable `VisitedList` from `visited_list_pool_`.

10 Neighbor Selection and Reciprocal Updates

The code does not simply connect to the nearest `M` candidates. It applies the HNSW neighbor diversification heuristic in `getNeighborsByHeuristic2`. Candidates are considered from nearest to farthest. A candidate is kept only if it is not closer to an already selected neighbor than it is to the query/new point. This tends to preserve angular or spatial diversity in the graph.

`mutuallyConnectNewElement` then:

- Writes the selected neighbors into the new node's link list at that level.
- For every selected neighbor, locks that neighbor's link list and tries to add the new node as a reciprocal connection.
- If the neighbor has free capacity, it appends the new node.
- If the neighbor is full, it builds a candidate set containing the old neighbors plus the new node, applies the same heuristic, and rewrites the neighbor list.

This means the graph is maintained as approximately bidirectional, but a full neighbor list can be pruned and rewritten when a new reciprocal edge is better according to the heuristic.

11 Detailed Control Flow of addPoint

The public insertion method is:

```
void addPoint(const void *data_point,
              labeltype label,
              bool replace_deleted = false)
```

This is the method called by the examples through statements such as:

```
alg_hnsw->addPoint(data + i * dim, i);
```

The pointer `data_point` is a generic pointer to the vector bytes that should be inserted. It is declared as `const void *` because the HNSW class does not need to know whether the vector came from a `float*`, `char*`, or another caller-owned byte buffer. The distance space supplies `data_size_` and the distance function, so `addPoint` can copy exactly the required number of bytes into the index.

11.1 Clarifications About Nodes, Layers, Edges, Memory Access, and Deletion

How the HNSW node data is stored. Each inserted multidimensional vector becomes one HNSW node. In the examples, one row of the original data matrix corresponds to one node. For example:

```
alg_hnsw->addPoint(data + i * dim, i);
```

Here `data + i * dim` is the caller's pointer to row `i`. The HNSW node does not keep this original pointer as the permanent node pointer. Instead, `addPoint` copies the vector bytes into HNSW-owned memory:

```
memcpy(getDataByInternalId(cur_c), data_point, data_size_);
```

Therefore, after insertion, the node's vector lives inside `data_level0_memory_`. The original array used by the example is only the input source. The index can still search after the caller's original array is gone, as long as the index itself has not been destroyed.

How one node is managed across multiple layers. One node can appear in multiple HNSW layers, but the vector data is not duplicated once per layer. The node has exactly one internal id `cur_c` and one stored vector. The layer membership is stored in:

```
element_levels_[cur_c] = curlevel;
```

Every node exists at layer 0. If `curlevel = 3`, then that same node id participates in layers 0, 1, 2, and 3. What changes from layer to layer is the node's neighbor list. Layer 0 neighbors are stored inside the node's main record in `data_level0_memory_`. Upper-layer neighbors are stored in `linkLists_[cur_c]`:

```
get_linklist0(cur_c) // layer 0 links
get_linklist(cur_c, level) // links for level >= 1
```

So the implementation does not create separate vector objects for the same node in different layers. It creates separate adjacency lists for the same internal node id at different layers.

How neighbors and connections are stored. Neighbors are stored as adjacency lists of internal node ids, not as linked-list objects and not as explicit edge pairs like `(1, 3)`. Each link list has this memory format:

```
[ neighbor_count ][ neighbor_id_0 ][ neighbor_id_1 ] ...
```

For example, if internal node 1 is connected to nodes 3, 8, and 10 on layer 0, then node 1's layer-0 link list stores a count of 3 followed by the internal ids 3, 8, and 10. There is no separate edge object. The reciprocal edge is represented by also placing 1 into the neighbor list of node 3, 8, or 10 when `mutuallyConnectNewElement` updates the graph.

How neighbor vectors are accessed during insertion and search. Neighbor exploration is pointer-based random access in memory. During construction or search, the graph traversal reads a neighbor id from the current node's adjacency list, then uses that id to compute the address of the neighbor's stored vector:

```
candidate_id = datal[j];
char *candidate_vector = getDataByInternalId(candidate_id);
dist = fstdistfunc_(query_or_new_vector,
                    candidate_vector,
                    dist_func_param_);
```

`getDataByInternalId(candidate_id)` performs address arithmetic:

```
data_level0_memory_
    + candidate_id * size_data_per_element_
    + offsetData_
```

Thus vectors are stored in one large contiguous memory block, but the visited neighbors are usually not visited in contiguous id order. The access pattern is graph-driven random access into `data_level0_memory_`. It does not load vectors from a file during normal insertion or search. Files are only used by `saveIndex` and `loadIndex` for persistence.

Why node deletion is needed. Deletion is needed because real indexes often need to remove or replace items after construction. Physically removing a node from an HNSW graph is expensive because many other nodes may have that internal id in their neighbor lists across multiple layers. This implementation therefore uses lazy deletion:

```
markDelete(label)
```

sets a delete bit in the node's level-0 link-list header. The node remains in the graph so traversal can still pass through it, but search results skip it. If `allow_replace_deleted_` is enabled, the deleted internal id is put into `deleted_elements` and can later be reused by `addPoint(..., replace_deleted = true)`. This saves memory and avoids growing the index capacity when old items are replaced by new items.

11.2 Public Wrapper: Replacement Policy

The first overload has two jobs: enforce the deleted-slot replacement policy and then dispatch to the internal insertion/update routine.

The first check is:

```
if ((allow_replace_deleted_ == false) && (replace_deleted == true)) {
    throw std::runtime_error("Replacement of deleted elements is disabled...");
}
```

This prevents a caller from requesting slot reuse unless the index was constructed with `allow_replace_deleted = true`. In normal insertion mode, `replace_deleted` is false, so the method simply locks the label and forwards to the internal overload:

```
std::unique_lock<std::mutex> lock_label(getLabelOpMutex(label));
if (!replace_deleted) {
    addPoint(data_point, label, -1);
    return;
}
```

`getLabelOpMutex(label)` selects one of the striped label locks. This prevents two threads from concurrently inserting, deleting, or replacing the same external label.

11.3 Replacement Path for Deleted Elements

If `replace_deleted` is true, the method tries to reuse an internal id from the `deleted_elements` set:

```
tableint internal_id_replaced;
std::unique_lock<std::mutex> lock_deleted_elements(deleted_elements_lock);
bool is_vacant_place = !deleted_elements.empty();
if (is_vacant_place) {
    internal_id_replaced = *deleted_elements.begin();
    deleted_elements.erase(internal_id_replaced);
}
lock_deleted_elements.unlock();
```

Here `internal_id_replaced` is an internal graph node id, not the external user label. The set stores nodes that were marked deleted earlier. The code removes one id from the set while holding `deleted_elements_lock` so that another thread cannot reuse the same slot.

If no deleted slot is available, the method falls back to normal insertion:

```
if (!is_vacant_place) {
    addPoint(data_point, label, -1);
}
```

If a deleted slot is available, the node storage is reused:

```
labeltype label_replaced = getExternalLabel(internal_id_replaced);
setExternalLabel(internal_id_replaced, label);

std::unique_lock<std::mutex> lock_table(label_lookup_lock);
label_lookup_.erase(label_replaced);
label_lookup_[label] = internal_id_replaced;
lock_table.unlock();

unmarkDeletedInternal(internal_id_replaced);
updatePoint(data_point, internal_id_replaced, 1.0);
```

This does not allocate a new node id. Instead, it changes the stored external label, updates the label lookup table, clears the deletion mark, and then calls `updatePoint`. `updatePoint` copies the new vector bytes into the old node's data region and repairs local graph connections.

11.4 Internal Overload: New Insert or Existing-Label Update

The actual graph insertion is handled by:

```
tableint addPoint(const void *data_point,
                  labeltype label,
                  int level)
```

The `level` argument is normally `-1`, meaning "sample a random HNSW level." Tests or special callers can pass a positive level to force a node level.

The method first locks `label_lookup_lock` and checks whether the external label already exists:

```
auto search = label_lookup_.find(label);
if (search != label_lookup_.end()) {
    tableint existingInternalId = search->second;
    ...
    updatePoint(data_point, existingInternalId, 1.0);
    return existingInternalId;
}
```

If the label already exists, no duplicate node is created. The existing node's vector is updated. If replacement of deleted elements is enabled, the method refuses to update an already-deleted node through this path because deleted nodes may be reused by the replacement path.

If the label is new, the method checks capacity:

```
if (cur_element_count >= max_elements_) {
    throw std::runtime_error("The number of elements exceeds the specified limit");
}
```

Then it assigns a dense internal id:

```
cur_c = cur_element_count;
cur_element_count++;
label_lookup_[label] = cur_c;
```

`cur_c` is the internal id used inside graph adjacency lists. `label` is the external id returned to users in search results.

11.5 Level Selection and Locking

After the internal id is reserved, the method locks the new node's link-list mutex:

```
std::unique_lock<std::mutex> lock_el(link_list_locks_[cur_c]);
```

It then chooses the node level:

```
int curlevel = getRandomLevel(mult_);
if (level > 0)
    curlevel = level;
element_levels_[cur_c] = curlevel;
```

The sampled value controls how many upper-layer link lists this node will own. Every node always has level 0. Nodes with `curlevel > 0` also have link lists for levels 1 through `curlevel`.

The method then takes the global lock and snapshots the current maximum graph level:

```
std::unique_lock<std::mutex> templock(global);
int maxlevelcopy = maxlevel_;
if (curlevel <= maxlevelcopy)
    templock.unlock();
```

If the new node is not higher than the current maximum level, the global lock can be released early. If the new node is higher, the lock is retained until the entry point and `maxlevel_` are updated.

11.6 Writing the Node's Stored Data

The method snapshots the current entry point:

```
tableint currObj = enterpoint_node_;
tableint enterpoint_copy = enterpoint_node_;
```

Then it initializes the new node's level-0 storage:

```
memset(data_level0_memory_ + cur_c * size_data_per_element_
      + offsetLevel0_, 0, size_data_per_element_);
```

The pointer expression computes the start address of node `cur_c`'s contiguous record in `data_level0_memory_`. That record contains:

```
[ level-0 links ][ vector bytes ][ external label ]
```

The label and vector are copied into their offsets:

```
memcpy(getExternalLabeLp(cur_c), &label, sizeof(labeltype));
memcpy(getDataByInternalId(cur_c), data_point, data_size_);
```

`getExternalLabeLp(cur_c)` returns a pointer to the label field inside the node record. `getDataByInternalId(cur_c)` returns a pointer to the vector data field. The source `data_point` remains owned by the caller; HNSW stores its own copy.

If the node has upper levels, the method allocates separate upper-layer link storage:

```
if (curlevel) {
    linkLists_[cur_c] =
        (char *) malloc(size_links_per_element_ * curlevel + 1);
    memset(linkLists_[cur_c], 0, size_links_per_element_ * curlevel + 1);
}
```

`linkLists_[cur_c]` is a pointer to a heap block containing link lists for levels 1 through `curlevel`. Level 0 is not stored here; it is stored in `data_level0_memory_`.

11.7 First-Node Case

If the graph is empty, `enterpoint_node_` was initialized to `-1`. The condition:

```
if ((signed)currObj != -1) {
    ...
} else {
    enterpoint_node_ = 0;
    maxlevel_ = curlevel;
}
```

therefore separates normal insertion from first-node initialization. The first node has no neighbors to connect to. It simply becomes the entry point and sets the maximum level.

11.8 Greedy Descent from Upper Layers

For every node after the first one, insertion starts from the current entry point. If the new node's level is lower than the current maximum level, the method greedily descends the upper layers:

```
for (int level = maxlevelcopy; level > curlevel; level--) {
    while (changed) {
        data = get_linklist(currObj, level);
        int size = getListCount(data);
        tableint *datal = (tableint *) (data + 1);
        ...
        if (d < curdist) {
            curdist = d;
            currObj = cand;
            changed = true;
        }
    }
}
```

The goal is to find a good entry point near the new vector before doing the more expensive construction search on lower layers. `get_linklist` returns the adjacency list for `currObj` at that upper level. `data + 1` skips the link-list count header, so `datal` points to the neighbor id array.

11.9 Construction Search and Mutual Connection

After greedy descent, the method connects the new node from `min(curlevel, maxlevelcopy)` down to level 0:

```
for (int level = std::min(curlevel, maxlevelcopy); level >= 0; level--) {
    auto top_candidates = searchBaseLayer(currObj, data_point, level);
    ...
    currObj = mutuallyConnectNewElement(data_point, cur_c,
                                        top_candidates, level, false);
}
```

`searchBaseLayer` performs an `ef_construction_`-wide candidate search around `currObj` at the current layer. The result is a priority queue of candidate neighbor internal ids.

`mutuallyConnectNewElement` then:

- applies the neighbor-selection heuristic,
- writes the selected neighbors into the new node's link list,
- inserts reciprocal links into those neighbor nodes,
- prunes a neighbor's link list if it is already full, and
- returns the next closest selected neighbor to use as the entry point for the next lower layer.

If the original entry point was marked deleted, the code explicitly adds it to the candidate queue so connectivity can still be repaired through it, while normal search results still avoid returning deleted nodes.

11.10 Entry Point Update

At the end of insertion, if the new node's level is higher than the previous maximum level, it becomes the new global entry point:

```
if (curlevel > maxlevelcopy) {
    enterpoint_node_ = cur_c;
    maxlevel_ = curlevel;
}
```

Finally, the method returns `cur_c`, the internal id assigned to the inserted or updated node.

11.11 Clarification: Difference Between addPoint and updatePoint

Main purpose. `addPoint` is the public insertion entry point. It decides whether the call should create a new node, update an existing label, or reuse a deleted node slot. `updatePoint` is not a general new-node insertion method. It assumes an internal node id already exists and replaces the vector stored at that internal id.

When addPoint is called. The examples and Python bindings call `addPoint` when they want to put data into the index:

```
alg_hnsw->addPoint(data + i * dim, i);
```

For a new label, `addPoint` reserves a new dense internal id, samples the node level, copies the vector and external label into HNSW-owned memory, allocates upper-layer link storage if needed, searches for candidate neighbors, and connects the new node into the graph.

When updatePoint is called. `updatePoint` is called from inside `addPoint` in two important cases:

- the caller inserts a label that already exists, so the existing node's vector should be updated instead of creating a duplicate node;
- `replace_deleted = true` reuses an internal id from a deleted node slot, so the old node storage receives a new vector and label.

In both cases, the internal id already exists before `updatePoint` starts.

Memory behavior. `addPoint` may allocate and initialize new graph memory. For a new node, it writes the level-0 record in `data_level0_memory_` and may allocate `linkLists_[cur_c]` for upper layers. `updatePoint` does not reserve a new internal id and does not create a new node record. Its first operation is:

```
memcpy(getDataByInternalId(internalId), dataPoint, data_size_);
```

That line overwrites the vector bytes of an already-existing node.

Graph behavior. `addPoint` connects a new node by searching from the entry point and calling `mutuallyConnectNewElement` layer by layer. `updatePoint` repairs graph neighborhoods around an existing node. It gathers one-hop and two-hop candidates around the updated node, recomputes neighbor lists for affected neighbors using the HNSW heuristic, and then calls `repairConnectionsForUpdate` to reconnect the updated point from the global entry point down to its own level.

Node id behavior. `addPoint` can increase `cur_element_count` when it creates a new node. `updatePoint` never increases `cur_element_count`. It keeps the same internal id and changes the stored vector, and sometimes the surrounding graph links are adjusted to match the new vector location in the metric space.

Short comparison.

Question	addPoint	updatePoint
Creates a new internal id?	Yes, for a new label.	No. Uses an existing internal id.
Copies vector bytes?	Yes.	Yes, overwrites existing vector bytes.
Allocates upper-layer link storage?	Yes, for a new node with level greater than 0.	No new node storage allocation.
Used for duplicate labels?	Detects the duplicate and dispatches to update.	Performs the actual vector update.
Used for deleted-slot replacement?	Selects and relabels a deleted slot.	Writes new vector data and repairs graph connections.
Changes graph links?	Builds links for a newly inserted node and reciprocal neighbors.	Repairs links around an existing node after its vector changes.

11.12 Short Logical Summary

The full insertion logic can be read as:

1. Validate whether deleted-slot replacement is allowed.
2. Lock by external label.
3. If replacement is requested, either reuse a deleted internal id or fall back to normal insertion.
4. In the internal overload, update an existing label if it already exists.
5. Otherwise reserve a new internal id and label mapping.
6. Sample or force the HNSW level.
7. Copy the external label and vector bytes into HNSW-owned memory.
8. Allocate upper-layer link storage if needed.
9. If this is the first node, make it the entry point.
10. Otherwise greedily descend upper layers, search candidate neighbors on each insertion layer, and mutually connect the new node.
11. If the new node has the highest level, make it the entry point.

12 Query Path

Querying is implemented by `HierarchicalNSW::searchKnn`.

The query path is:

1. Start at `enterpoint_node_`.
2. Compute the distance from the query to the entry point.
3. For levels `maxlevel_` down to 1, greedily move to any neighbor that is closer to the query. This is a single-best path descent on the sparse upper layers.
4. At level 0, call `searchBaseLayerST` with `max(ef_, k)`.
5. Trim the candidate heap down to `k` results.
6. Convert internal ids back to external labels before returning.

The base-layer search scans adjacency lists from the candidate queue. For each visited node it reads the layer-0 neighbor list, computes distances to unvisited neighbors, and updates the candidate queue and result heap. If there are no deletions and no filter function, the implementation uses a `bare_bone_search` template path to avoid deletion/filter checks.

13 Original Data Handling

The original input vectors are copied into the index at insertion time. The key line is the `memcpy` in `addPoint`:

```
memcpy(getDataByInternalId(cur_c), data_point, data_size_);
```

For Python:

- Input data is accepted as a C-style NumPy array and force-cast to `float`.
- For `l2` and `ip`, the row bytes are copied directly into HNSW memory.
- For `cosine`, the binding normalizes each vector first and stores the normalized vector, not the original unnormalized vector.

The method `get_items` retrieves vectors by external label through `getDataByLabel`. For cosine indexes, this returns the normalized stored vectors because those are the bytes kept by the HNSW structure.

14 Index Save File and Load File Format

`saveIndex(location)` writes a binary file containing both metadata and graph/data memory. The order is:

1. `offsetLevel0_`
2. `max_elements_`
3. `cur_element_count`
4. `size_data_per_element_`
5. `label_offset_`
6. `offsetData_`
7. `maxlevel_`
8. `enterpoint_node_`
9. `maxM_`
10. `maxM0_`
11. `M_`
12. `mult_`
13. `ef_construction_`
14. Raw `data_level0_memory_` for all current elements.
15. For each element, the size of its upper-layer link block followed by that block's bytes.

This file stores the vectors, labels, level-0 graph links, upper-layer graph links, entry point, levels, and construction parameters needed to reconstruct the index. On load, `loadIndex` reconstructs `label_lookup_` by reading each stored external label from `data_level0_memory_`. Query-time `ef_` is reset to 10 when loading through the C++ `loadIndex` path; the README notes that users should set `ef` manually after loading a saved index.

15 Deletion and Replacement

Deletion is lazy. `markDelete(label)` maps the label to an internal id and sets a delete bit inside the level-0 link-list header. It does not physically remove the node from the graph. Searches skip marked nodes in results, but traversal may still pass through them.

If `allow_replace_deleted_` is true, deleted internal ids are stored in `deleted_elements`. A later insertion with `replace_deleted=True` may reuse one of those internal ids, replace the stored label and vector, clear the delete bit, and call `updatePoint` to repair local graph connections.

16 Locking Mechanism During Construction and Deletion

This section clarifies why locks are used while constructing the HNSW graph, what `lock_deleted_elements` means, and how deletion/un-deletion interacts with locking.

16.1 Why Locks Are Required During HNSW Graph Construction

HNSW construction modifies shared graph state. When a new node is inserted, `addPoint` does more than copy a vector. It also reserves an internal id, updates the external-label lookup table, writes the node's level-0 memory record, allocates upper-layer link storage, searches candidate neighbors, writes the new node's neighbor lists, and updates reciprocal neighbor lists in already-existing nodes.

If two threads insert nodes at the same time, they may touch overlapping structures:

- `label_lookup_` maps external labels to internal ids.
- `cur_element_count` assigns the next internal node id.
- `data_level0_memory_` stores each node's level-0 links, vector bytes, and label.
- `linkLists_` stores upper-layer link lists.
- `enterpoint_node_` and `maxlevel_` define the global top-level entry point.
- Existing nodes' adjacency lists may be rewritten when reciprocal edges are added.

Without locks, two insertions could assign the same label mapping, update the same neighbor list at the same time, corrupt a link-list count, or change the entry point while another insertion is using it. The implementation therefore uses different locks for different shared resources:

- `getLabelOpMutex(label)` locks operations involving one external label.
- `label_lookup_lock` protects the label-to-internal-id table.
- `link_list_locks_[id]` protects the adjacency lists of a particular internal node.
- `global` protects global entry-point and maximum-level changes.
- `deleted_elements_lock` protects the set of reusable deleted node ids.

The important design idea is that the graph is not locked with one single large lock for every operation. Instead, the code locks smaller pieces of state, such as one label, one node's link list, or the deleted-id set. This allows more concurrency while still protecting memory that can be modified by multiple threads.

16.2 Meaning of lock_deleted_elements

The statement:

```
std::unique_lock<std::mutex> lock_deleted_elements(deleted_elements_lock);
```

creates a scoped lock object named `lock_deleted_elements`. The mutex being locked is `deleted_elements_lock`. The protected data structure is:

```
std::unordered_set<tableint> deleted_elements;
```

`deleted_elements` stores internal node ids that were marked deleted and are available for possible replacement. The lock is needed because one thread may mark a node deleted while another thread is trying to reuse a deleted slot in `addPoint(..., replace_deleted = true)`.

In the replacement path, the code checks and removes a reusable id while holding this lock:

```
bool is_vacant_place = !deleted_elements.empty();
if (is_vacant_place) {
    internal_id_replaced = *deleted_elements.begin();
    deleted_elements.erase(internal_id_replaced);
}
lock_deleted_elements.unlock();
```

This prevents two insertions from taking the same deleted internal id. After one thread erases `internal_id_replaced` from the set, no other thread can pick that same id as a vacant slot.

16.3 When a Node Is Marked Deleted

A node is marked deleted when the user calls:

```
markDelete(label)
```

The public method first locks operations for that external label:

```
std::unique_lock<std::mutex> lock_label(getLabelOpMutex(label));
```

Then it locks `label_lookup_lock` long enough to translate the external label into an internal id:

```
auto search = label_lookup_.find(label);
tableint internalId = search->second;
lock_table.unlock();
```

After that, `markDeletedInternal(internalId)` sets the deletion bit:

```
unsigned char *ll_cur = ((unsigned char *)get_linklist0(internalId)) + 2;
*ll_cur |= DELETE_MARK;
num_deleted_ += 1;
```

The deletion mark is stored inside the level-0 link-list header, not in a separate node object. This is why deletion is described as lazy: the node is not physically removed from the graph, and its neighbor links are not erased. Search can still traverse through the node, but it should not return the node as a result.

If replacement is enabled, the internal id is inserted into `deleted_elements`:

```
if (allow_replace_deleted_) {
    std::unique_lock<std::mutex> lock_deleted_elements(deleted_elements_lock);
    deleted_elements.insert(internalId);
}
```

Here `deleted_elements_lock` protects the reusable-id set while the deleted node id is added.

16.4 When a Node Is Unmarked Deleted

A node is unmarked when the user calls:

```
unmarkDelete(label)
```

The public path is similar to deletion: lock by label, find the internal id in `label_lookup_`, then call `unmarkDeletedInternal`.

The internal method clears the deletion bit:

```
unsigned char *ll_cur = ((unsigned char *)get_linklist0(internalId)) + 2;
*ll_cur &= ~DELETE_MARK;
num_deleted_ -= 1;
```

If replacement is enabled, the node must also be removed from the reusable deleted-id set:

```
if (allow_replace_deleted_) {
    std::unique_lock<std::mutex> lock_deleted_elements(deleted_elements_lock);
    deleted_elements.erase(internalId);
}
```

This lock prevents a race where one thread unmarks the node while another thread is trying to reuse it as a deleted slot.

16.5 Relationship Between Deletion, Un-Deletion, and Replacement

The three operations share the same state:

- the delete bit inside the node's level-0 link-list header,
- `num_deleted_`, which counts deleted nodes, and
- `deleted_elements`, which stores reusable internal ids when replacement is enabled.

Marking deletion sets the bit and optionally inserts the internal id into `deleted_elements`. Unmarking deletion clears the bit and optionally erases the internal id from `deleted_elements`. Replacement insertion removes an id from `deleted_elements`, assigns a new external label to that same internal id, clears the deletion mark, copies new vector bytes into the old node storage, and repairs local connections with `updatePoint`.

The locking relationship is therefore:

- label locks prevent conflicting operations on the same external label,
- `label_lookup_lock` protects label-to-internal-id lookup and updates, and
- `deleted_elements_lock` protects the pool of deleted ids that can be reused.

This is why `lock_deleted_elements` is specifically required: the deleted-id pool is shared by delete, un-delete, and replacement insertion.

17 Threading and Locks

The implementation is CPU multithreaded at the Python batch level. The binding uses `ParallelFor` to distribute rows across threads for `add_items` and `knn_query`. Inside the HNSW class, thread safety is handled with several lock groups:

- `label_op_locks_`: striped locks for operations on external labels.
- `label_lookup_lock`: protects the label-to-internal-id map.
- `link_list_locks_`: one mutex per internal node for link list updates and reads during construction/update.
- `global`: protects global entry-point and maximum-level changes during insertion.
- `deleted_elements_lock`: protects the reusable deleted-id set.

The README states that `add_items` is thread-safe with other `add_items` calls, and `knn_query` is thread-safe with other `knn_query` calls. However, `add_items` and `knn_query` are not advertised as thread-safe with each other.

18 Mechanism of the C++ Examples

This section explains the runnable C++ examples in `examples/cpp`. All of these examples create synthetic random data, build an in-memory HNSW index, and then exercise a particular query or maintenance mode. They are small demonstrations of the API rather than full benchmark programs.

18.1 Basic Search: example_search.cpp

The basic search example demonstrates the normal HNSW build/query/save/load cycle.

1. It sets the dataset shape and construction parameters: `dim = 16`, `max_elements = 10000`, `M = 16`, and `ef_construction = 200`.
2. It creates an L2 distance space: `hnswlib::L2Space space(dim)`.
3. It creates the graph index: `HierarchicalNSW<float>(&space, max_elements, M, ef_construction )`.
4. It allocates one flat array, `float* data = new float[dim * max_elements]`. The pointer `data` owns a contiguous block of all generated vectors. Vector `i` begins at `data + i * dim`.
5. It inserts every vector with `alg_hnsw->addPoint(data + i * dim, i)`. The second argument `i` is the external label.
6. It queries each vector against the index with `searchKnn(data + i * dim, 1)`. Because the query vector is one of the stored vectors, the expected nearest label is `i`.
7. It counts how often the returned top label equals the query label and prints recall.
8. It saves the index with `saveIndex("hnsw.bin")`, deletes the old index object, reloads from the saved file, and repeats the recall check.

Mechanically, `searchKnn` first performs greedy descent through the upper HNSW layers and then performs a wider candidate search on layer 0. The returned object is a `std::priority_queue` of `(distance, label)` pairs, where `result.top()` is used to read the best label for `k = 1`.

18.2 Filtered Search: example_filter.cpp

The filter example demonstrates label-based filtering during search. It defines:

```
class PickDivisibleIds: public hnswlib::BaseFilterFunctor {
    bool operator()(hnswlib::labeltype label_id) {
        return label_id % divisor == 0;
    }
};
```

This class is a callable filter object. For `PickDivisibleIds pickIdsDivisibleByTwo(2)`, only even labels are allowed.

The index construction is the same as the basic search example: random vectors are generated, copied into the HNSW index with `addPoint`, and labels are assigned from 0 to `max_elements - 1`. The difference is the query call:

```
searchKnnCloserFirst(data + i * dim, k, &pickIdsDivisibleByTwo)
```

The third argument is a pointer to the filter functor. During base-layer search, hnswlib can still traverse graph nodes to preserve connectivity, but a candidate is only accepted into the returned result set if `isIdAllowed(label)` returns true. The example checks every returned label and prints an error if any odd label appears.

`searchKnnCloserFirst` is a convenience wrapper around `searchKnn`. It converts the priority-queue result into a vector ordered from closest to farthest, which is easier to inspect in a loop.

18.3 Multithreaded Search: example_mt_search.cpp

The multithreaded search example uses the same HNSW index type, but distributes insertion and querying across CPU threads. The file defines a local `ParallelFor` helper.

The helper works as follows:

- If `numThreads <= 0`, it uses `std::thread::hardware_concurrency()`.
- If `numThreads == 1`, it runs a normal serial loop.
- Otherwise, it creates `numThreads` worker threads.
- The shared atomic counter `current` hands out row ids using `current.fetch_add(1)`.
- Each worker calls the supplied lambda for one row at a time.
- Exceptions are captured in `lastException` and rethrown after all threads join.

Insertion is parallelized with:

```
ParallelFor(0, max_elements, num_threads, [&](size_t row, size_t threadId) {
    alg_hnsw->addPoint((void*)(data + dim * row), row);
});
```

Here `data + dim * row` is a pointer to the first float of the row's vector. The cast to `void*` matches the generic `addPoint` API, which accepts raw vector bytes. The graph object uses internal locks to make concurrent insertions safe with other insertions.

Querying is also parallelized. Each thread calls `searchKnn(data + dim * row, 1)` for a different row and writes the returned label into `neighbors[row]`. After all threads finish, the main thread computes recall by comparing `neighbors[i]` with `i`.

This example should be read as "parallel batch insert, then parallel batch query." It does not mix insertion and query at the same time.

18.4 Multithreaded Filtered Search: example_mt_filter.cpp

The multithreaded filtered example combines the two previous mechanisms:

- It uses the same `ParallelFor` worker-thread helper as `example_mt_search.cpp`.
- It uses the same `PickDivisibleIds` filter class as `example_filter.cpp`.
- It inserts all vectors in parallel.
- It searches all vectors in parallel with a filter pointer.

The query call is:

```
searchKnn(data + dim * row, k, &pickIdsDivisibleByTwo)
```

The output of `searchKnn` is a priority queue, so the example pops `k` labels from the queue and stores them into a flat result vector:

```
neighbors[row * k + i] = label;
```

This memory layout means query row `row` owns the result segment `neighbors[row * k]` through `neighbors[row * k + k - 1]`. After the parallel query phase, the main thread scans all stored labels and reports an error if a returned label is not divisible by two.

Filtering generally makes search slower than the bare query path because the implementation cannot use the simplest no-deletion/no-filter template branch. It must check whether each candidate label is allowed before accepting it into the result heap.

18.5 Epsilon Search: example_epsilon_search.cpp

The epsilon search example demonstrates a custom stop condition rather than a fixed `k`-nearest-neighbor query. It uses:

```
hnswlib::EpsilonSearchStopCondition<dist_t>
    stop_condition(epsilon2, min_num_candidates, max_elements);

alg_hnsw->searchStopConditionClosest(query_data, stop_condition);
```

The example stores data in a raw byte array:

```
size_t data_point_size = space.get_data_size();
char* data = new char[data_point_size * max_elements];
```

The pointer `data` owns all vector records as bytes. For vector `i`, `point_data = data + i * data_point_size` points to the start of that record. Each coordinate is written by computing a byte offset, casting the address to `float*`, and assigning a float value:

```
char* vec_data = point_data + j * sizeof(float);
*(float*)vec_data = value;
```

`epsilon2` is the maximum squared L2 distance for final results. `min_num_candidates` forces the search to inspect at least a minimum number of candidates before stopping just because the next candidate is outside the epsilon radius. `max_elements` is passed as the maximum number of candidates, so this example allows the stop condition to return as many matching vectors as needed up to the dataset size.

Inside `EpsilonSearchStopCondition`, search can stop when either:

- the candidate distance is worse than the current lower bound and the maximum candidate count has been reached, or
- the candidate distance is larger than `epsilon2` and at least `min_num_candidates` have already been collected.

After the graph search finishes, `filter_results` removes candidates whose distance is greater than `epsilon2`. The example prints how many vectors were found for each random query.

18.6 Multivector Search: example_multivector_search.cpp

The multivector example demonstrates searching for documents where each document may be represented by multiple vectors. The key difference is the space type:

```
hnswlib::MultiVectorL2Space<docidtype> space(dim);
```

This space stores a normal float vector plus an extra document id field at the end of each data record. Therefore:

```
data_point_size = dim * sizeof(float) + sizeof(docidtype)
```

conceptually, although the code obtains this value through `space.get_data_size()`. A point record looks like:

```
[ float vector coordinates ][ doc_id ]
```

The example writes the vector coordinates manually into a raw `char*` buffer and then calls:

```
space.set_doc_id(point_data, doc_id);
```

`set_doc_id` writes the document id at byte offset `vector_size_`, immediately after the vector coordinates. During index construction, each vector still receives its own HNSW label, and the example keeps a separate `label_docid_lookup` map so it can count which returned labels belong to which documents.

The query mechanism uses:

```
hnswlib::MultiVectorSearchStopCondition<docidtype, dist_t>
    stop_condition(space, num_docs, ef_collection);

alg_hnsw->searchStopConditionClosest(query_data, stop_condition);
```

`num_docs` is the target number of distinct documents to return. `ef_collection` is the number of distinct candidate documents to maintain during search; larger values usually improve recall but slow the search. The stop condition counts documents, not just individual vectors. It uses the space object to read each candidate vector's stored document id from the point bytes. It can stop once the next candidate cannot improve the result and `ef_collection` distinct documents have already been collected.

After the result vectors are returned, the example maps each returned label back to a document id and prints both the number of distinct documents found and the number of vectors returned.

19 Distance-Space Implementation: space_ip.h and space_l2.h

The two headers `hnswlib/space_ip.h` and `hnswlib/space_l2.h` implement the CPU distance kernels used by both graph construction and query. They do not construct or traverse the HNSW graph themselves. Instead, a space object describes the size of one vector record and supplies HNSW with a function pointer having the following generic signature:

```
using DISTFUNC = MTYPE (*)(const void*, const void*, const void*);
```

The first two pointers identify the two vector records to compare. The third pointer identifies the distance-function parameter; for these spaces it points to the stored dimension `dim_`. HNSW obtains and retains these values through `get_data_size()`, `get_dist_func()`, and `get_dist_func_param()`. Therefore the graph algorithm can call the selected metric without knowing whether the records contain floats or bytes, or whether the selected implementation is scalar, SSE, AVX, or AVX-512.

These are CPU implementations. SSE, AVX, and AVX-512 are CPU vector instruction sets; no CUDA kernel, GPU allocation, or host-to-device transfer occurs in either header. The preprocessor macros `USE_SSE`, `USE_AVX`, and `USE_AVX512` are established in `hnswlib.h` from compiler target macros. The constructors then use `AVXCapable()` and `AVX512Capable()` to avoid selecting an instruction set that the running CPU and operating system cannot support.

19.1 Mechanism of space_ip.h

Metric represented by the file. `InnerProductSpace` exposes inner-product similarity as a distance:

$$d_{\mathrm{IP}}(x, y) = 1 - \sum_{i=0}^{D-1} x_i y_i.$$

HNSW is written to prefer smaller distances, so a larger dot product must be converted into a smaller value. The value can be negative when the dot product is greater than one; this is valid for the ordering used by HNSW. For unit-normalized vectors, minimizing this distance gives the same ranking as maximizing cosine similarity. Without normalization it ranks by raw inner product, not cosine similarity.

Scalar path. `InnerProduct` casts both generic vector addresses to `float*`, reads the dimension from `qty_ptr`, multiplies corresponding coordinates, and accumulates their sum. The wrapper `InnerProductDistance` returns one minus that sum. This scalar function is the constructor's initial and portable fallback choice.

SIMD kernels. The header contains kernels for dimensions processed in blocks of 4 or 16 floats:

- SSE uses 128-bit registers, so one load contains four floats.
- AVX uses 256-bit registers, so one load contains eight floats; the 16-element kernels issue two such loads per block.
- AVX-512 uses 512-bit registers, so one load contains sixteen floats.

Each loop performs vector loads from both input addresses, element-wise multiplication, and vector accumulation. The accumulator lanes are reduced to one scalar dot product at the end. The loads use unaligned intrinsics such as `_mm_loadu_ps` and `_mm256_loadu_ps`, so the caller is not

required to align every stored vector to a SIMD boundary. Temporary reduction arrays are explicitly aligned for aligned stores.

Residual-coordinate handling. When the dimension is not divisible by the selected block width, a residual wrapper divides the work into a SIMD prefix and scalar tail. For example, `InnerProductDistanceSIMD16ExtResiduals` computes `floor(D/16)*16` coordinates with the selected 16-wide kernel, advances both float pointers by that many coordinates, and computes the remaining `D mod 16` coordinates with `InnerProduct`. Only after combining both partial dot products does it apply `1.0f - sum`.

Constructor dispatch. `InnerProductSpace(dim)` first records the scalar function. If SIMD was compiled in, it updates the shared SIMD function pointers to the strongest runtime-supported implementation and chooses the final distance function from the dimension:

Dimension condition	Selected logical path
`dim % 16 == 0`	16-element SIMD kernel
`dim % 4 == 0`	4-element SIMD kernel
`dim > 16`	16-element SIMD prefix plus scalar tail
`dim > 4`	4-element SIMD prefix plus scalar tail
otherwise	scalar inner product

The selected width describes how coordinates are grouped, not necessarily the physical register width. For example, the 16-element AVX implementation processes each block as two 8-float AVX operations.

Class state and pointer lifetime. `dim_` is the coordinate count, `data_size_` is `dim * sizeof (float)`, and `fstdistfunc_` stores the selected function address. `get_dist_func_param()` returns `&dim_` rather than a copy, so the `InnerProductSpace` object must remain alive as long as its HNSW index uses that pointer. The vector pointers passed to a distance call refer either to the query buffer or to a vector copied into the index's level-zero node record; the distance code reads those coordinates but does not own, allocate, free, or modify either vector.

19.2 Mechanism of space_l2.h

Metric represented by the file. `L2Space` computes squared Euclidean distance:

$$d_{\mathrm{L2}}(x, y) = \sum_{i=0}^{D-1} (x_i - y_i)^2.$$

It deliberately does not call `sqrt`. Squared distance has the same nearest-neighbor ordering as Euclidean distance and avoids an unnecessary square root for every comparison. Distances returned by searches are therefore squared L2 values.

Scalar float path. `L2Sqr` casts the two generic addresses to `float*`, dereferences `qty_ptr` to obtain the dimension, and walks through both contiguous arrays. At each coordinate it computes the difference, squares it, and adds it to a float accumulator. Incrementing `pVect1` and `pVect2` advances by one float, not one byte.

SIMD float paths. `L2SqrSIMD16ExtSSE`, `L2SqrSIMD16ExtAVX`, and `L2SqrSIMD16ExtAVX512` process 16-coordinate blocks using the available register width. They load coordinates from both vectors, subtract the register values, square the differences by multiplying each difference by itself, and accumulate the lanes. The SSE-only `L2SqrSIMD4Ext` handles dimensions divisible by four. As in the inner-product file, unaligned vector loads allow index records and external query buffers to be compared without a strict input-alignment contract.

Residual-coordinate handling. `L2SqrSIMD16ExtResiduals` computes the largest prefix divisible by 16 and sends the tail to scalar `L2Sqr`. `L2SqrSIMD4ExtResiduals` does the same with a four-coordinate prefix. In contrast to inner product, the partial results are already distances, so the wrapper simply returns `SIMD_prefix + scalar_tail`.

Constructor dispatch. `L2Space(dim)` uses the same dimension decision order as `InnerProductSpace` : exact 16-coordinate SIMD blocks first, then exact 4-coordinate blocks, then a 16-wide or 4-wide SIMD prefix with a scalar tail, and finally fully scalar computation for very small dimensions. When AVX-512 code is compiled and supported at runtime, the 16-wide pointer is set to the AVX-512 kernel. Otherwise AVX is selected when compiled and supported; the pointer remains on the SSE implementation when neither higher option is usable.

For the current `dim = 3584` data,

$$3584 \bmod 16 = 0,$$

so `L2Space(3584)` chooses `L2SqrSIMD16Ext` and no scalar residual loop is needed. The actual implementation behind that pointer is AVX-512, AVX, or SSE according to the build flags and runtime CPU checks.

Byte-vector L2 implementation. The lower part of the same header also defines `L2SpaceI` for `unsigned char` vector records and an integer distance result. It selects the manually unrolled `L2SqrI4x` when the byte dimension is divisible by four; otherwise it uses `L2SqrI`. This is separate from the float `L2Space` used by the `.fvecs` example. The "I" implementation is loop unrolling over byte values, not an SSE/AVX intrinsic kernel.

Class state and pointers. For float L2, `data_size_` equals `dim * sizeof(float)`. For byte L2 it equals `dim * sizeof(unsigned char)`. In both classes, `fstdistfunc_` is a callable function pointer and `get_dist_func_param()` returns the address of the member `dim_`. The `const void*` inputs are non-owning, read-only addresses; casts give the kernel a typed view of existing bytes and do not copy the vectors.

19.3 How the Distance Space Is Used by HNSW

The typical setup is:

```
hnswlib::L2Space space(dim);
hnswlib::HierarchicalNSW<float> index(&space, max_elements,
                                      M, ef_construction);
```

During `HierarchicalNSW` construction, the index stores the result of `space.get_dist_func()` in its distance-function member and stores the result of `space.get_dist_func_param()` as the associated parameter pointer. During insertion, greedy layer descent, construction search, neighbor pruning, and query search, it repeatedly calls that same function pointer with two vector addresses and the dimension pointer. Consequently, changing from L2 to inner product changes the distance calculations and therefore can change graph edges as well as query rankings; it is not merely an output-format choice.

The indexed vector is accessed through a pointer into the index's in-memory node storage. A query is accessed through the pointer supplied to `searchKnn`. Coordinates within either one vector are contiguous, which lets a SIMD kernel stream through them efficiently. Neighbor vectors may occupy scattered node records, so graph traversal can cause irregular loads between distance calls, but each individual distance call reads one contiguous vector from each selected record.

19.4 Controlling SSE, AVX, and AVX-512

Both compile-time availability and runtime capability affect dispatch. A compiler option such as `-march=native` normally enables every instruction set supported by the build machine, after which runtime checks choose the strongest compiled implementation supported by the executing machine. A portable SSE-targeted build can be made explicitly, for example:

```
g++ -O3 -std=c++11 -msse2 -mno-avx -mno-avx512f -I. \
    examples/cpp/example_search_w_data_source.cpp \
    -o build_cpp/example_search_sse
```

An AVX build may use `-mavx -mno-avx512f`, while an AVX-512 build may use `-mavx512f` on a compatible compiler and CPU. Defining `NO_MANUAL_VECTORIZATION` before including hnswlib disables these manual SIMD paths and leaves the scalar function selected. Binaries explicitly compiled for an unsupported ISA can fail before hnswlib's runtime dispatch protects a particular kernel, because the compiler may emit that ISA elsewhere in the program. For portable deployment, build for the oldest target CPU or produce separate binaries for each target.

19.4.1 Summary for the Current Machine

The output of `lscpu` on the current machine reports SSE, AVX, AVX2, and AVX-512 support, including the `avx512f` foundation feature used by these hnswlib kernels. Therefore, when the example is compiled with:

```
g++ -O3 -std=c++11 -march=native -I. \
    examples/cpp/example_search_w_data_source.cpp \
    -o build_cpp/example_search_w_data_source
```

`-march=native` causes the compiler to define the AVX-512 feature macro for this host. At runtime, `AVX512Capable()` confirms that the CPU and operating system can execute and preserve AVX-512 register state. The dispatch priority is:

$$\text{AVX-512} \longrightarrow \text{AVX} \longrightarrow \text{SSE} \longrightarrow \text{scalar fallback.}$$

The arrows mean "fall back to the next implementation when the preceding one was not compiled or is not supported at runtime." For the current `dim = 3584` dataset, `3584 mod 16 = 0`. Thus there is no scalar residual tail, and a native build selects:

- `L2SqrSIMD16ExtAVX512` for `hnswlib::L2Space(3584)`; and
- `InnerProductDistanceSIMD16ExtAVX512` for `hnswlib::InnerProductSpace(3584)`.

Although the CPU also reports AVX2, these headers do not define a separately named AVX2 distance kernel. The manually written choices are SSE, AVX, and AVX-512. The compiler may still use AVX2 or FMA instructions when optimizing other code under `-march=native`.

For controlled comparisons, separate binaries can be compiled as follows:

```
# SSE only
g++ -O3 -std=c++11 -msse2 -mno-avx -mno-avx512f -I. \
    examples/cpp/example_search_w_data_source.cpp \
    -o build_cpp/example_search_sse

# AVX, with AVX-512 disabled
```

```
g++ -O3 -std=c++11 -mavx -mno-avx512f -I. \
    examples/cpp/example_search_w_data_source.cpp \
    -o build_cpp/example_search_avx

# AVX-512
g++ -O3 -std=c++11 -mavx512f -I. \
    examples/cpp/example_search_w_data_source.cpp \
    -o build_cpp/example_search_avx512
```

In short, the default for the present `-march=native` build on this machine is AVX-512. It would commonly be SSE for a generic x86-64 build that does not enable AVX or AVX-512 compiler targets, and it becomes scalar when `NO_MANUAL_VECTORIZATION` is defined.

20 Constructing HNSW from the Compressed CPRQ Package Without Full Decompression

This section analyzes how the W-encoding package can be used to construct and query an HNSW graph without first materializing the complete `rows * cols` float matrix. It describes possible designs and their tradeoffs; it does not imply that stock hnswlib already accepts this package directly.

20.1 What the Package Actually Stores

The package is one sequential `std::vector<uint8_t>` containing three logical regions:

```
[ fixed header ]
[ quantized mean ][ quantized basis ][ quantized alpha ]
[ encoded residual byte vector ]
```

The fixed header stores the magic number, version, matrix dimensions, fixed-length residual block size, absolute error bound, rank, oversampling, maximum residual magnitude, residual block counts and sizes. Every `QuantizedBlock` then stores:

```
float min_val
float max_val
uint32_t rows
uint32_t cols
uint8_t bits
uint64_t value_count
uint8_t values[value_count]
```

The last region starts with a `uint64_t` element count because it is written by:

```
cprq_kmeans_package_append_vector<uint8_t>
```

The encoded residual bytes follow that count.

This is a serialized container, not a row-major array of vectors. In particular, the bytes belonging to one reconstructed row are distributed among:

- the shared quantized mean;
- the shared quantized basis;
- one row of quantized alpha; and
- one residual entry in every dimension-wise residual block.

Consequently, the address `bytes.data()+ row * cols` is not the address of row `row`. Passing `bytes.data()` to `hnswlib::L2Space` would make the L2 kernel reinterpret package headers and compressed codes as `float` coordinates. The result would be mathematically invalid and could also read beyond the intended region.

20.2 Reconstruction Model and Exact L2 Distance

For the global mean-rSVD package, a database row is represented approximately as:

$$\widehat{x}_i = \mu + B\alpha_i + e_i,$$

where μ is the dequantized mean, B is the dequantized `cols * rank` basis, α_i is row i's dequantized rank-dimensional coefficient vector, and e_i is its decoded `cols`-dimensional residual.

An 8-bit quantized model value q is reconstructed using:

$$v(q) = v_{\min} + q\frac{v_{\max} - v_{\min}}{2^b - 1}.$$

The residual decoder first locates the logical block for a dimension and a row block. A zero-block bitmap can immediately imply a residual value of zero. For a nonzero block, the decoder reads its bit-width header, one packed sign bit, and the packed magnitude for the row. The decoded value is:

$$e_{i,d} = s_{i,d} m_{i,d} \Delta_e, \qquad \Delta_e = 2\,\epsilon_{\text{abs}},$$

where $s_{i,d}$ is $+1$ or -1, $m_{i,d}$ is the decoded integer magnitude, and ϵ_{abs} is the package error bound.

For two database nodes, the shared mean cancels:

$$\widehat{x}_i - \widehat{x}_j = B(\alpha_i - \alpha_j) + (e_i - e_j).$$

Therefore an exact squared-L2 calculation over reconstructed data is:

$$d(i,j) = \sum_{d=0}^{D-1} \left[\sum_{r=0}^{R-1} B_{d,r}(\alpha_{i,r} - \alpha_{j,r}) + e_{i,d} - e_{j,d} \right]^2.$$

This equation can be evaluated without creating either complete float row. It still has substantial work per comparison: alpha dequantization or lookup, residual decoding across all dimensions, low-rank multiplication, and accumulation.

For a raw query q, query preprocessing can compute:

$$q_c = q - \mu, \qquad \alpha_q = B^T q_c, \qquad r_q = q_c - B\alpha_q.$$

The exact query-to-database distance is then:

$$d(q,i) = \|B(\alpha_q - \alpha_i) + (r_q - e_i)\|_2^2.$$

This is the same accuracy-first mechanism represented by the existing path:

```
EXACT_ROW_RQ_RECONSTRUCTION
```

in `compressed_direct_ann_search.cu`. It reconstructs the difference one dimension at a time inside the distance calculation rather than materializing all database vectors.

20.3 Residual Random Access Is a Required Index

The residual stream is dimension-major and block-oriented. A logical block is approximately:

```
logical_block = dimension * blocks_per_dimension + row_block
```

for the global model. Nonzero blocks have variable payload lengths because their magnitude bit widths may differ. Therefore a decoder cannot generally jump to an arbitrary row residual using a simple fixed byte stride.

Before HNSW construction, the compressed stream should be parsed once to create read-only lookup metadata such as:

- a nonzero-block bitmap;
- a header index or nonzero index for every logical block;
- payload byte or bit offsets for every nonzero block;
- sign-stream bit offsets;
- magnitude-stream bit offsets; and
- original-to-reordered row mappings when a KMeans layout is used.

This metadata is an index over compressed bytes, not decompression of the matrix. It enables approximately constant-time location of one encoded residual value. The existing compressed-direct CUDA path already builds and uses this kind of metadata.

The present package reader constructs new `std::vector` objects and copies each serialized region. A future memory-mapped or zero-copy loader would need a bounds-checked view parser that returns pointer-and-length spans into the package. It must keep the mapped file alive for the entire HNSW lifetime. Adding serialized section offsets and optional serialized residual lookup offsets would reduce startup parsing and make random access cleaner.

20.4 Why Stock L2Space Cannot Store the Compressed Rows

`HierarchicalNSW` obtains a fixed `data_size_` from its `SpaceInterface`. Every `addPoint` copies exactly that many bytes into the level-zero node record. Standard `L2Space(D)` requires those bytes to be exactly `D * sizeof(float)` contiguous float coordinates.

The CPRQ representation violates that assumption in two ways:

1. one row does not occupy one fixed contiguous package record; and
2. a distance requires shared model state and residual lookup metadata.

Thus compressed-resident HNSW requires a custom `SpaceInterface<float>` or changes to the HNSW distance-call interface. The graph structure itself does not need to change: graph edges can still store dense internal node ids exactly as they do now.

20.5 Option 1: Stream One Reconstructed Row into Ordinary HNSW

The least invasive method is:

1. parse the package and dequantize the shared model;
2. allocate one reusable `cols`-float scratch row;
3. decode and reconstruct row i into that scratch row;
4. call `addPoint(scratch.data(), i)`; and
5. reuse the scratch storage for row $i + 1$.

This avoids a separate full `rows * cols` decompressed input matrix. However, it is important to state what hnswlib then does: `addPoint` copies every reconstructed row into the HNSW index. After construction, the index still contains the complete float matrix in its level-zero records.

Advantages.

- Requires no change to hnswlib's graph or distance code.
- Uses the existing highly optimized SSE/AVX/AVX-512 L2 kernels.
- Produces the same graph that would be produced from the same fully reconstructed lossy matrix, subject to insertion order and threading.
- Query execution remains fast because every stored row is contiguous.
- Existing index serialization and loading continue to work normally.

Disadvantages.

- Does not preserve compressed database memory after graph construction.
- The saved HNSW file includes all reconstructed float vectors and can be much larger than the CPRQ package.
- Construction performs reconstruction once per row and then retains another full float copy inside HNSW.
- Multithreaded insertion needs one scratch row per worker thread.

This option satisfies "no full temporary decompression" but not "keep node vectors compressed inside the final HNSW index."

20.6 Option 2: Store Row Descriptors and Decode Distances on Demand

A truly compressed-resident design can make each HNSW node store a small fixed-size descriptor instead of a float vector. The simplest descriptor is a row id:

```
struct CompressedPointRef {
    uint32_t row_id;
};
```

The custom space would report `sizeof(CompressedPointRef)` as its data size. During graph construction, `addPoint` would receive one descriptor for every row, and hnswlib would copy only the row id into its node record. Its custom distance function would use that id to access alpha and residual codes through an immutable compressed-package context passed through `get_dist_func_param()`.

The distance context would need at least:

- package base addresses and validated section lengths;
- rows, columns, rank, quantization scales, and residual step;
- dequantized or quantized mean and basis representation;
- alpha layout and row mapping;
- residual random-access tables; and
- thread-local scratch or caches when a kernel requires temporary data.

For construction, both function inputs are row descriptors. For query, one input must represent a raw query or a preprocessed query state while the other is a stored row descriptor. This can be implemented with a tagged, fixed-size query/node descriptor, but a pointer embedded in a query descriptor is non-owning and is valid only for the synchronous search call. A cleaner long-term change is a HNSW interface that has distinct node-to-node and query-to-node distance callbacks.

Advantages.

- HNSW node payloads can be four or eight bytes instead of `cols * sizeof(float)`.
- The complete reconstructed matrix is never resident.
- The CPRQ package remains the authoritative vector store.
- Exact distance to the reconstructed lossy data is possible.
- Graph serialization can be much smaller when it stores descriptors and edges but not float rows.

Disadvantages.

- Requires a custom space and probably a cleaner query-distance API.
- Every HNSW comparison may decode scattered residual fields and perform low-rank arithmetic, while graph construction performs many comparisons.
- HNSW traversal visits irregular node ids, which reduces cache locality and makes compressed block batching difficult.
- Standard `space_l2.h` SIMD kernels cannot operate directly on a row-id descriptor.
- Multithreading requires immutable package state, thread-safe lookup tables, and per-thread scratch buffers.
- A saved graph is no longer self-contained: it must be reopened with the exact matching package, row ordering, model version, and dimensions.
- Existing hnswlib index files would need a format/version identifier for the custom payload semantics.

This design minimizes memory, but it is likely to make construction and query substantially slower unless compressed-distance kernels, caching, and the storage layout are carefully redesigned.

20.7 Option 3: Build HNSW on Compact Surrogate Codes and Rerank

A practical compromise is to use each row's rank-dimensional alpha vector as the HNSW node vector. The graph then uses:

$$d_\alpha(i, j) = \|\alpha_i - \alpha_j\|_2^2.$$

If B has orthonormal columns, $\|B(\alpha_i - \alpha_j)\|_2^2 = \|\alpha_i - \alpha_j\|_2^2$, so this exactly captures the low-rank component. It omits the residual difference and its cross term with the low-rank component. Quantization may also weaken perfect orthogonality.

At query time, HNSW retrieves a larger candidate set using alpha-space distance. The system then evaluates the exact compressed-domain distance for those candidates and returns the best k.

The candidate count can be controlled by `ef` or by explicitly requesting more than k neighbors before reranking.

An improved surrogate can append a compact residual sketch, product quantization code, block norm summary, or projected residual features to alpha. This increases graph payload size but may recover candidates whose similarity is dominated by residual structure.

Advantages.

- Graph construction uses low-dimensional contiguous data and can use ordinary SIMD L2 kernels.
- Node memory becomes approximately `rank * sizeof(float)` instead of `cols * sizeof(float)`.
- Exact residual decoding is performed only for the candidate set, not for every graph edge comparison.
- Query latency can be tuned with the HNSW candidate count.
- The design is simpler and usually much faster than decoding an exact full-dimensional distance for every HNSW comparison.

Disadvantages.

- HNSW graph topology is based on a surrogate metric.
- True neighbors can be missed before reranking, and reranking cannot recover candidates that HNSW never returned.
- Recall depends on rank, error bound, residual energy, surrogate features, M, `ef_construction`, and query `ef`.
- The graph and package must retain consistent row ids.
- Accuracy must be measured against exact search over the same reconstructed data and, separately, against the original data if original data quality is the final objective.

For most CPU hnswlib integrations, this is the recommended first design: construct on alpha or alpha plus a residual sketch, retrieve a generous candidate set, and rerank those candidates using the exact compressed formula.

20.8 Option 4: Block-Level Graph or Two-Level Index

Because residual bytes are organized by row blocks, a graph node can represent a block rather than one row. A block descriptor can contain a centroid or compact summary. HNSW first finds promising blocks, then a blockwise decoder evaluates all rows in those blocks.

Advantages.

- Matches the residual compression layout and improves sequential access.
- Decoding one block amortizes headers and bit-unpacking over many rows.
- SIMD or GPU execution can process multiple rows together.
- The top-level graph has fewer nodes.

Disadvantages.

- It is not a conventional one-node-per-vector HNSW index.
- Block summaries may hide a relevant row inside an apparently distant block.
- Candidate expansion, block scoring, and final row-level top-k require new logic outside stock hnswlib.
- Recall depends strongly on block construction and the number of blocks searched.

This option can be attractive when compressed-block throughput matters more than preserving the exact hnswlib interface.

20.9 Exact, Fast, and Corrected Compressed Distances

The existing compressed-direct search code identifies three useful distance modes.

Exact reconstructed difference.

$$\|B(\alpha_q - \alpha_i) + (r_q - e_i)\|_2^2$$

retains all terms and should match L2 search over explicitly reconstructed lossy rows, within floating-point and decoder consistency.

Fast separable approximation.

$$\|\alpha_q - \alpha_i\|_2^2 + \|r_q - e_i\|_2^2$$

uses basis orthonormality for the low-rank norm but omits the cross term. It is cheaper but can change ordering.

Beta-corrected approximation. The existing code also describes a correction using projections of residuals onto the basis. This restores the missing cross contribution more closely when the required beta values are precomputed, at the cost of extra rank-dimensional storage and arithmetic.

For HNSW graph construction, using different metrics for construction and query is allowed technically but affects recall. An alpha-only or fast distance graph should therefore be treated as a candidate generator, followed by exact reranking.

20.10 Can SIMD Be Used?

Yes, but not by calling `L2SqrSIMD16ExtAVX512` directly on the serialized package. SIMD is useful inside specific stages.

Quantized model dequantization. Arrays of 8-bit mean, basis, and alpha values are contiguous. SSE, AVX2, or AVX-512BW can widen batches of bytes to integer lanes, convert them to float, and apply:

$$v_{\min} + q \cdot \text{scale}.$$

The shared mean and basis should normally be dequantized once and retained, because their memory cost is $D + DR$, not ND. Alpha can either be retained as bytes and dequantized per accessed row or dequantized once into NR floats. The latter is only a partial decompression and is much smaller than ND when $R \ll D$.

Low-rank arithmetic. The operations $B\alpha$, $B^T q$, alpha-space L2, and the exact dimension-wise difference contain regular multiply-add loops. AVX2 or AVX-512 with FMA can process multiple basis entries or dimensions per iteration. Layout matters: a basis stored as `basis[dimension][rank]` favors computing one output dimension from one alpha row, while a transposed or blocked copy may improve projection and batch throughput.

Residual bit unpacking. SIMD can help, but this stage is less regular. Zero-block tests, variable bit widths, cross-byte fields, sign bits, and scattered block offsets introduce branches and gathers. Useful techniques include:

- process an entire row block instead of one arbitrary row;
- group blocks by bit width;
- unpack fixed-width magnitudes into integer vectors;
- apply sign masks with vector blend or XOR operations;
- use lookup tables for small bit widths; and
- accumulate distances for several rows in parallel.

AVX-512 is especially useful when lanes represent several rows of the same residual block, because the encoded block already groups neighboring rows. This resembles the existing CUDA design, where one warp lane handles one row in a 32-row block. It fits brute-force or blockwise reranking better than ordinary HNSW, whose distance requests arrive as irregular pairs of row ids.

Reconstruct-then-SIMD scratch path. One distance function can decode a candidate row into a thread-local contiguous float buffer and then call an AVX-512 L2 kernel against a contiguous query. This uses SIMD for the final subtraction and squaring, but decoding and low-rank reconstruction may dominate the cost. It is more effective when a decoded row is cached and reused across several comparisons.

Alpha-only HNSW path. If graph construction uses dequantized alpha vectors, the dimension is the rank rather than the original column count. Standard hnswlib SIMD dispatch works immediately. Padding the rank to a multiple of 16 can avoid a residual SIMD tail, provided padded coordinates are zero and the stored dimension is consistently treated as the padded dimension.

20.11 Why a Per-Distance GPU Call Is Not Appropriate

The package already has CUDA kernels that calculate compressed distances without full matrix reconstruction. However, stock hnswlib performs graph traversal on the CPU and requests one or a few distances at a time. Launching a CUDA kernel and synchronizing for each HNSW comparison would normally cost more than the useful arithmetic.

GPU use becomes effective when work is batched:

- brute-force compressed search over many rows;
- blockwise candidate scoring;
- batched reranking of HNSW candidates from many queries;
- offline batched neighbor generation; or
- a fully GPU-resident graph implementation with GPU traversal.

Therefore a CPU HNSW plus GPU reranking design should collect candidate ids first and transfer or score them in batches. A fully GPU HNSW would require a different graph implementation; changing only `space_l2.h` does not move hnswlib construction or traversal to the GPU.

20.12 Caching and Multithreading Considerations

Exact on-demand decoding should use immutable package bytes and immutable lookup tables so all HNSW construction threads can read them concurrently. Mutable scratch vectors, reconstructed rows, and temporary alpha values must be thread-local or protected.

Possible caches include:

- a small per-thread cache of recently reconstructed rows;
- a shared read-only dequantized mean and basis;
- dequantized alpha rows, if NR float storage is acceptable;
- residual block decode caches keyed by dimension and row block; and
- precomputed row norms, residual norms, or beta correction vectors.

A shared mutable LRU cache would need synchronization and may become a construction bottleneck. Per-thread caches use more memory but avoid locks. The correct cache strategy should be chosen from measured HNSW access traces, because graph insertion and query may have different locality.

20.13 Persistence and Validation Requirements

A compressed-resident graph must be bound to its source package. The saved graph should record or verify:

- package magic and version;
- rows, columns, rank, quantization bits, and residual block size;
- modeling path and row-reordering semantics;
- a strong package checksum or content hash;
- distance mode used during graph construction;
- descriptor format version; and
- HNSW M, `ef_construction`, and random seed.

Without these checks, loading a graph with a different package can silently make every row id refer to the wrong vector.

Validation should compare at least:

1. one-row on-demand reconstruction against the existing full decompressor;
2. compressed exact distance against L2 over two reconstructed rows;
3. compressed query distance against L2 over a reconstructed row;

4. graph recall against exact search over reconstructed data;

5. end-to-end recall against original uncompressed data; and

6. construction time, query QPS, peak memory, package bytes, graph bytes, and reranking cost.

Exact comparisons should allow a small floating-point tolerance. Row mapping tests are essential for KMeans packages because compressed rows may be stored in cluster-reordered order while labels are expected in original order.

20.14 Recommended Development Sequence

The following sequence isolates correctness risk while preserving a path to a compressed-resident design:

1. Implement and validate a parser plus residual random-access tables.

2. Implement one-row reconstruction into a scratch buffer and compare it with the existing full decompressor.

3. Build the baseline streaming-row ordinary HNSW to establish recall, construction time, and QPS over reconstructed data.

4. Build an alpha-space HNSW using row ids as labels.

5. Retrieve more than k candidates and exact-rerank them directly from compressed components.

6. Measure recall as candidate count, rank, M, `ef_construction`, and query `ef` change.

7. Only if memory requirements demand it, implement a descriptor-only custom HNSW space and compare its exact on-demand distance cost against the surrogate-and-rerank design.

8. Add CPU SIMD block decoding or GPU batch reranking only after profiling identifies the dominant stage.

20.15 Overall Recommendation

It is possible to construct HNSW without fully decompressing the CPRQ matrix, but the meaning of "without full decompression" must be chosen explicitly.

- For minimum implementation effort and maximum query speed, reconstruct one row at a time and let ordinary HNSW retain float rows.

- For minimum final memory, store row descriptors and calculate exact compressed distances on demand, accepting major implementation complexity and likely lower throughput.

- For the strongest practical balance, build HNSW on dequantized alpha vectors or alpha plus a residual sketch, then exact-rerank a larger candidate set from compressed residuals.

- For maximum compressed decoding throughput, use a block-level or GPU batched design rather than invoking a GPU kernel per HNSW distance.

Given the current dimension $D = 3584$, a rank such as $R = 64$, and the existing CPU hnswlib implementation, the alpha-HNSW plus exact compressed reranking design is the recommended starting point. It reduces graph vector payload by approximately $D/R = 56$ times before graph-link overhead, retains SIMD-friendly contiguous graph vectors, avoids reconstructing the complete database, and confines the expensive residual decoding to a tunable candidate set.

21 Proposed HNSW Nodes with Row-Wise Signed Residual Bins, Alpha, and Beta

This section analyzes a more direct design: decode the dimension-major, bit-packed residual stream once into row-major signed quantization bins, store those integer bins as the HNSW node payload, and combine their integer residual distance with row-wise low-rank alpha and precomputed beta terms. The design is feasible, especially for mean-centered global rSVD, but several details are necessary for mathematical correctness and for compatibility with hnswlib.

21.1 Restatement of the Compressed Model

Let X_i be original row i, let μ be the dimension-wise mean, and let B be the low-rank basis. The compressor first forms:

$$X_{c,i} = X_i - \mu$$

and a low-rank approximation:

$$A_i = B\alpha_i.$$

The unquantized residual is:

$$e_i = X_{c,i} - B\alpha_i.$$

With absolute error bound ϵ, define the residual quantization step:

$$\Delta = 2\epsilon.$$

The signed integer bin is:

$$z_{i,d} = \text{round}\left(\frac{e_{i,d}}{2\epsilon}\right) = \text{round}\left(\frac{e_{i,d}}{\Delta}\right).$$

The parentheses are important. In C or C++, the expression:

```
residual / 2 * eb
```

means (residual/2)ϵ, which is not the intended quantizer. The implementation must use:

```
round(residual / (2.0f * absolute_error_bound))
```

The reconstructed lossy row is:

$$\widehat{X}_i = \mu + B\alpha_i + \Delta z_i.$$

The proposal does not reconstruct and store $\widehat{X}_i$ as D floats. It stores z_i as D signed integers and keeps the model terms required to calculate distance.

21.2 From Dimension-Major Blocks to Row-Major Node Codes

The encoded residual stream groups 32 rows for one dimension. For each logical block:

- the global block mask says whether all 32 values are zero;
- a zero block contributes 32 zero bins and has no payload;
- a nonzero block has a one-byte header containing magnitude bit width;
- packed sign bits identify positive and negative bins; and
- packed magnitude bits store the absolute integer bin values.

Data preparation can allocate a row-major matrix Z initialized to zero:

$$Z \in \mathrm{Z}^{N \times D}.$$

It then traverses the original dimension-major blocks. Zero blocks require no write beyond the initialization. For each nonzero block, it decodes the sign and magnitude of each valid lane and writes:

$$Z[\mathrm{row_start} + \mathrm{lane}, d] = \begin{cases} -m, & \text{negative}, \\ +m, & \text{positive}. \end{cases}$$

The resulting row `Z[i][0:D]` is contiguous and can be copied into one HNSW node.

Meaning of partial decompression. This procedure avoids floating-point reconstruction of the $N \times D$ matrix, but it does decode every residual bin and expands the variable-bit stream into a fixed-width $N \times D$ integer matrix. It is therefore best described as *full residual-code unpacking without float decompression*, not as decoding only a small subset of residuals. The final HNSW index keeps the expanded integer codes.

21.3 Selecting Signed 8-Bit or Signed 16-Bit Storage

Residual bins are signed. A plain `uint8_t` is not sufficient unless an explicit bias or zigzag representation is used and reversed before subtraction. The simplest node representation is:

- `int8_t` when every signed bin is in $[-127, 127]$;
- `int16_t` when every signed bin is in $[-32767, 32767]$; and
- `int32_t` or rejection/recompression when either bound is exceeded.

Let $m_{\max}$ be the maximum decoded magnitude across all nonzero blocks. The actual values should be checked during transposition:

$$w = \begin{cases} 8, & m_{\max} \le 127, \\ 16, & 127 < m_{\max} \le 32767, \\ 32, & m_{\max} > 32767. \end{cases}$$

The block-header bit width provides a conservative preliminary check. If every magnitude header is at most 7 bits, `int8_t` is safe. If every header is at most 15 bits, `int16_t` is safe. An

8-bit *magnitude* can encode 255, so it does not fit in a signed 8-bit node value. Likewise, a 16-bit magnitude does not fit in `int16_t`.

Stock hnswlib requires one fixed `data_size_` for all nodes. Therefore the simplest implementation chooses one code width globally for the whole index. Mixing 8-bit and 16-bit nodes would require a fixed tagged union, padding every node to the largest format, or changing hnswlib's storage model.

21.4 Global State and Per-Node State

For mean-centered global rSVD, the following state can be shared by every node:

- D, rank R, and residual step Δ;
- the dimension-wise mean μ;
- the reconstructed global basis B, or an orthonormalized basis described later;
- quantization metadata and selected residual-code width;
- package checksum, version, and row-order mapping; and
- HNSW metric-mode metadata.

A self-contained node payload can contain:

```
NodePayload:
    row_id
    optional cluster_id
    coefficient[R] // alpha or transformed alpha
    residual_beta[R] // beta or transformed beta
    residual_bin[D] // int8_t or int16_t
```

Keeping coefficient, beta, and bins inside the node gives one pointer to all distance operands and makes serialization straightforward. Alternatively, the node can contain only row id plus residual bins while alpha and beta remain in global row-major arrays. That reduces the bytes copied into `data_level0_memory_` but introduces additional random memory loads. In either layout, alpha and beta exist once per row; calling them "global" means globally managed arrays, not one shared value for all rows.

The mean and basis do not need to be stored in every node. The mean cancels from database-to-database distance, and the basis is needed globally for query preparation and beta construction.

21.5 Beta Computed by the Existing CUDA Method

The method:

```
cprq_prepare_database_residual_beta_mean_centered_global_rsvd
```

decodes one residual value at a time and computes, for each database row:

$$\beta_i = B^T \widehat{e}_i = B^T(\Delta z_i),$$

where $\widehat{e}_i = \Delta z_i$ is the reconstructed quantized residual. The output has shape $N \times R$. It is a persistent row-wise float matrix, not an $N \times D$ reconstructed residual matrix.

The current CUDA kernel launches over row blocks and rank components. For each (i, r), it traverses dimensions, skips zero residual blocks, decodes the signed magnitude, multiplies by $B_{d,r}$, and accumulates $\beta_{i,r}$. The proposed row-major bin preparation could calculate beta during the same preparation phase or in a later matrix operation over ΔZ.

21.6 Database-to-Database Distance Derivation

For two HNSW nodes i and j, define:

$$\delta\alpha = \alpha_i - \alpha_j, \qquad \delta z = z_i - z_j, \qquad \delta\beta = \beta_i - \beta_j.$$

Because the common mean cancels:

$$\widehat{X}_i - \widehat{X}_j = B\delta\alpha + \Delta\delta z.$$

Expanding squared L2 gives:

$$d(i, j) = \delta\alpha^T(B^T B)\delta\alpha\Delta^2\|\delta z\|_2^2 2\delta\alpha^T\delta\beta.$$

The integer residual term is exactly the proposed expression:

$$\Delta^2\|\delta z\|_2^2 = 4\epsilon^2 \sum_{d=0}^{D-1}(z_{i,d} - z_{j,d})^2.$$

The beta identity follows because:

$$\delta\beta = B^T(\Delta z_i) - B^T(\Delta z_j) = B^T(\Delta\delta z).$$

Thus $2\delta\alpha^T\delta\beta$ is exactly the cross term between low-rank and residual differences.

21.7 Relation to the Current Corrected CUDA Distance

The active beta-corrected search kernel computes:

$$d_{\text{current}} = \|\delta\alpha\|_2^2\|\delta e\|_2^2 2\delta\alpha^T\delta\beta.$$

This equals the fully expanded reconstructed-vector distance when:

$$B^T B = I.$$

The original rSVD basis is intended to be orthonormal, but quantization and dequantization can make the reconstructed basis only approximately orthonormal. Query beta helps restore the cross term; it does not correct the coefficient norm $\delta\alpha^T B^T B\delta\alpha$. Therefore there are three implementation choices:

1. Use the current formula and accept a small approximation when $B^T B$ is close to identity.
2. Store the global Gram matrix $G = B^T B$ and evaluate $\delta\alpha^T G\delta\alpha$, costing approximately R^2 operations per distance.
3. Reparameterize the basis once with thin QR, which preserves $O(D + R)$ distance and can make the formula exact.

21.8 Recommended Exact QR Reparameterization

Compute a thin QR factorization of the dequantized basis:

$$B = QT,$$

where $Q^TQ = I$ and T is an $R \times R$ upper-triangular matrix. For every row, define:

$$a_i = T\alpha_i, \qquad \gamma_i = Q^T(\Delta z_i).$$

The reconstructed vector is unchanged:

$$B\alpha_i = Q(T\alpha_i) = Qa_i.$$

The exact database-to-database distance becomes:

$$d(i,j) = \|a_i - a_j\|_2^2\Delta^2\|z_i - z_j\|_2^2 2(a_i - a_j)^T(\gamma_i - \gamma_j).$$

This has $O(D + R)$ work, the same asymptotic cost as the proposed corrected formula, but no $B^TB \approx I$ assumption. Beta must be recomputed against Q, so the existing beta produced against B cannot simply be relabeled as γ.

Numerical roundoff can produce a tiny negative final value after adding the cross term. Values in a small tolerance below zero can be clamped to zero. A materially negative value indicates an overflow, inconsistent beta, an incorrect sign convention, or a broken orthonormalization invariant.

21.9 Query-to-Node Distance

A raw float query should normally remain unquantized. For the QR design, preprocess it once:

$$q_c = q - \mu, \qquad a_q = Q^Tq_c, \qquad r_q = q_c - Qa_q, \qquad \gamma_q = Q^Tr_q.$$

In exact arithmetic, $\gamma_q = 0$ because r_q is orthogonal to Q, but retaining or checking it is useful for floating-point validation. The exact query-to-node distance is:

$$d(q,i) = \|a_q - a_i\|_2^2\|r_q - \Delta z_i\|_2^2 2(a_q - a_i)^T(\gamma_q - \gamma_i).$$

The database-to-database residual term can use integer differences and the factor $4\epsilon^2$. The raw-query residual term mixes float r_q with integer z_i, so it must convert bins to Δz_i during accumulation.

Quantizing the query residual as:

$$z_q = \text{round}(r_q/\Delta)$$

would allow the integer formula $\Delta^2\|z_q - z_i\|^2$, but it adds query quantization error and can change neighbor ordering. It should be an optional measured approximation, not the default correctness path.

21.10 Required Change: Keep Every Query in Full Precision

The recommended design decision is to *not quantize the query.* Database rows are already represented by the compressor's lossy reconstruction:

$$\widehat{X}_i = \mu + Qa_i + \Delta z_i,$$

but the query remains its original FP32 vector q. Search therefore ranks:

$$\|q - \widehat{X}_i\|_2^2,$$

not:

$$\|\widehat{q} - \widehat{X}_i\|_2^2$$

for a separately quantized query $\widehat{q}$. This avoids a second source of quantization error and is especially important near distance ties, where a small query perturbation can change graph traversal and final ranking.

This does not recover information already lost in the database compression. Results are still neighbors of the lossy reconstructed database when measured against a full-precision query. Evaluation should report recall against both the reconstructed database and the original database so that query handling and database compression error are not confused.

21.11 Minimal-Change Integration Without Modifying hnswalg.h

The current hnswlib call pattern permits a full-precision query without changing `hnswalg.h`:

- `addPoint` copies exactly `data_size_` bytes from each database node into HNSW storage;
- graph construction calls the distance function with two node payload pointers;
- `searchKnn` does not copy `query_data` into the graph; and
- every search comparison passes the original `query_data` pointer as the first distance operand and a stored node pointer as the second operand.

A custom space can exploit this by using two tagged operand formats:

```
enum OperandKind : uint32_t {
    STORED_QUANTIZED_NODE,
    FULL_PRECISION_QUERY
};

StoredNode:
    kind = STORED_QUANTIZED_NODE
    row_id
    transformed_alpha[R]
    residual_gamma[R]
    signed_residual_bin[D]

FullPrecisionQueryDescriptor:
    kind = FULL_PRECISION_QUERY
    pointer_to_fp32_query[D]
    pointer_to_query_alpha[R]
    pointer_to_query_residual[D]
    pointer_to_query_gamma[R]
```

Only `StoredNode` is passed to `addPoint` and copied into the HNSW index. The smaller query descriptor is passed only to `searchKnn`. Its size does not have to equal the stored `data_size_` because hnswlib never copies the query pointer.

The custom distance function dispatches on the first operand:

```
CompressedDistance(left, right, context):
    if left.kind == FULL_PRECISION_QUERY:
        return FullPrecisionQueryToNodeDistance(left, right, context)
    else:
        return QuantizedNodeToNodeDistance(left, right, context)
```

During graph construction, both operands are stored-node payloads, so the integer-bin node-to-node formula is selected. During search, the first operand is always the temporary FP32 query descriptor and the second operand is a stored node, so the full-precision query formula is selected.

Important safety rule. A full-precision query descriptor must never be passed to `addPoint`. HNSW would attempt to copy the larger stored-node byte count from that address and could read invalid memory. The two operations should be exposed through separate strongly typed wrapper methods so this mistake cannot occur accidentally.

21.12 Full-Precision Query Preparation and Distance

For the mean-centered QR representation, prepare one query as:

$$q_c = q - \mu, \qquad a_q = Q^T q_c, \qquad r_q = q_c - Q a_q, \qquad \gamma_q = Q^T r_q.$$

All of these calculations use FP32 or, if numerical testing justifies it, FP64 accumulation. No rounding by Δ is applied to q, r_q, or a_q.

For stored node i, calculate:

$$d(q,i) = \|a_q - a_i\|_2^2 \sum_{d=0}^{D-1} (r_{q,d} - \Delta z_{i,d})^2 + 2(a_q - a_i)^T(\gamma_q - \gamma_i).$$

This expression equals $\|q - \widehat{X}_i\|_2^2$ for the QR representation. Only the database residual is converted from an integer bin. The query residual remains float throughout the comparison.

The query descriptor should normally retain pointers to:

- the caller's original contiguous FP32 query;
- a thread-local R-float transformed query coefficient;
- a thread-local D-float query residual; and
- a thread-local R-float query gamma vector.

For $D = 3584$ and $R = 64$, this temporary state is only about 14.5 KiB per simultaneous query when stored as FP32, dominated by the residual vector. It is released or reused after `searchKnn` returns and is never added to the persistent graph.

21.13 Lifetime and Multithreaded Query Rules

`searchKnn` is synchronous in the current CPU implementation, so descriptor pointers need to remain valid only until that call returns. A safe wrapper owns the temporary vectors on its stack or in thread-local reusable buffers, creates the descriptor after preprocessing, calls `searchKnn`, and destroys or reuses the buffers afterward.

For concurrent queries:

- the mean, basis, Δ, and other model context remain immutable and shared;
- every worker has its own query descriptor and query scratch vectors;
- no current-query pointer is stored in the shared space object; and
- the distance callback reads query state only through its first operand.

Storing a mutable "current query" pointer in `get_dist_func_param()` would be unsafe because simultaneous searches could overwrite it. Passing query state through `query_data` avoids that race.

21.14 Full-Precision Query Pseudocode with Minimal Changes

```
SearchFullPrecision(query_fp32, k):
    scratch.a = transpose(Q) * (query_fp32 - mean)
    scratch.r = (query_fp32 - mean) - Q * scratch.a
    scratch.gamma = transpose(Q) * scratch.r

    descriptor.kind = FULL_PRECISION_QUERY
    descriptor.query = query_fp32
    descriptor.a = scratch.a.data()
    descriptor.r = scratch.r.data()
    descriptor.gamma = scratch.gamma.data()

    return hnsw_index.searchKnn(&descriptor, k)

DistanceCallback(left, right, context):
    if left.kind == FULL_PRECISION_QUERY:
        node = cast_stored_node(right)
        return squared_L2(left.a, node.a)
             + squared_float_to_scaled_integer_residual(
                  left.r, node.Z, context.Delta)
             + 2 * dot(
                  left.a - node.a,
                  left.gamma - node.gamma)

    node_left = cast_stored_node(left)
    node_right = cast_stored_node(right)
    return corrected_integer_node_distance(node_left, node_right)
```

The original FP32 query pointer is included for validation, debugging, or an alternative direct formula. The optimized corrected distance primarily uses the precomputed FP32 query components.

21.15 SIMD with a Full-Precision Query

Keeping the query in FP32 still permits SIMD:

1. load 16 signed 8-bit or 16-bit database bins;
2. widen them safely to integer lanes;
3. convert the lanes to FP32;
4. multiply by Δ;
5. subtract them from 16 FP32 query-residual values; and
6. accumulate squared differences with AVX-512 multiply-add operations.

The transformed alpha and gamma terms are already FP32 and can use the same AVX-512 rank-vector path as before. Thus full-precision queries do not require full database reconstruction and do not prevent vectorization. They replace the database-to-database integer residual subtraction with a float-query-to-scaled-integer subtraction only during search.

21.16 Per-Query Signed-Bin Distance Lookup Tables

A per-query lookup table can avoid repeatedly converting each database bin to float and evaluating a subtraction and square. For query residual coordinate $r_{q,d}$, residual step Δ, sign s, and magnitude m, define:

$$T^{+}_{d,m} = (r_{q,d} - \Delta m)^2,$$

$$T^{-}_{d,m} = (r_{q,d} + \Delta m)^2.$$

For a stored signed bin $z_{i,d}$, the residual contribution is selected as:

$$C_{i,d} = \begin{cases} T^{+}_{d,|z_{i,d}|}, & z_{i,d} \geq 0, \\ T^{-}_{d,|z_{i,d}|}, & z_{i,d} < 0. \end{cases}$$

The full query residual distance is:

$$\|r_q - \Delta z_i\|^2 = \sum_{d=0}^{D-1} C_{i,d}.$$

This is mathematically exact for the FP32 query residual and the reconstructed integer database residual. It does not quantize the query.

21.16.1 Alignment and Mixed FP32/Integer Types

The query residual and database bins do not need to have the same element type or occupy the same address alignment. A direct AVX-512 arithmetic kernel loads them separately:

1. load signed 8-bit or 16-bit database bins;
2. widen them to 32-bit integer lanes;

3. convert those lanes to FP32;

4. multiply by Δ;

5. load the corresponding FP32 query residual lanes; and

6. subtract, square, and accumulate.

Sixty-four-byte alignment is beneficial but not required. The kernel can use unaligned load intrinsics, as the current hnswlib distance kernels do. This is important because the address of one HNSW node depends on its complete record stride, graph-link bytes, and data offset; merely aligning the residual array inside the node structure does not guarantee that every node begins at a 64-byte boundary.

The lookup-table path changes the SIMD problem. It replaces regular conversion and multiply-add instructions with indexed memory reads. AVX2 and AVX-512 gather instructions can perform several lookups, but gathers from widely separated table rows may be limited by cache and memory latency. A table being 64-byte aligned does not solve a table-footprint or random-access problem.

21.16.2 Full Per-Query Table Size

If one positive and one negative FP32 table are allocated for every dimension and every magnitude from zero through $M_{\max}$, the memory requirement is:

$$S_{\text{table}} = 2D(M_{\max} + 1)\text{sizeof(float)}.$$

For $D = 3584$:

- $M_{\max} = 127$ requires approximately 3.5 MiB per query;
- $M_{\max} = 255$ requires approximately 7.0 MiB per query; and
- $M_{\max} = 32767$ requires approximately 896 MiB per query.

The first two sizes may fit in a shared last-level cache but normally do not fit in L1 or L2. The 16-bit worst-case table is not practical. Concurrent queries multiply this memory requirement by the number of search threads.

The table also costs:

$$O(DM_{\max})$$

work to build for every query. This cost can be amortized when HNSW performs many distance calculations for the same query, but it may dominate a low-`ef` search or a query that terminates early.

21.16.3 Multiple Dimension-Wise Positive and Negative Tables

Instead of allocating one rectangular table using the same global $M_{\max}$ for every dimension, the query can own two logical cache tables for each dimension:

$$T_d^+[0 \dots M_d], \qquad T_d^-[0 \dots M_d],$$

where M_d is the largest database residual magnitude observed in dimension d. Their entries are:

$$T_d^+[m] = (r_{q,d} - \Delta m)^2,$$

$$T_d^-[m] = (r_{q,d} + \Delta m)^2.$$

For node i, dimension d reads:

```
magnitude = abs(node.Z[d])
if node.Z[d] >= 0:
    contribution = positive_table[d][magnitude]
else:
    contribution = negative_table[d][magnitude]
```

This is still an exact FP32-query-to-quantized-database residual distance. The query itself remains unquantized.

Variable-size memory requirement. The logical table size becomes:

$$S_{\text{dimension tables}} = 2\,\text{sizeof(float)} \sum_{d=0}^{D-1} (M_d + 1).$$

This can be much smaller than $2D(M_{\max} + 1)$sizeof(float) when only a few dimensions contain large residual bins. If most dimensions reach the same maximum, there is little or no saving.

Recommended physical layout. Creating $2D$ independent heap allocations is usually undesirable. For $D = 3584$, that would mean 7168 small allocations per active query. It adds allocator overhead, pointer storage, fragmentation, and extra pointer chasing. A better implementation uses:

```
positive_values[sum_d(M_d + 1)]
negative_values[sum_d(M_d + 1)]
positive_offset[D + 1]
negative_offset[D + 1]
```

Each dimension is a separate *logical* table, but all positive entries occupy one contiguous pool and all negative entries occupy another. An offset array locates the start of dimension d. The two offset arrays can be identical when positive and negative tables have equal lengths.

An alternative interleaves the two signs for each dimension:

```
[positive dimension d][negative dimension d]
```

This places both sign choices near each other in memory. Separate positive and negative pools may be better when signs are highly skewed and one pool is rarely accessed. Both layouts should be benchmarked rather than selected from alignment alone.

Alignment tradeoff. Aligning every dimension table to a 64-byte cache-line boundary can simplify SIMD loads and prevent two dimensions from sharing a cache line, but it adds padding. In the worst case, thousands of short tables can waste hundreds of kilobytes per query. Packed tables minimize memory, while aligning only larger or frequently used tables is a reasonable hybrid.

Pros of dimension-wise sign-separated tables.

- Uses actual M_d rather than the largest magnitude in the whole dataset.
- Avoids reserving unused magnitude entries for low-range dimensions.

- Positive and negative formulas are precomputed exactly.
- Sign handling is simple: select one base pointer and use magnitude as the index.
- A dimension's tables may remain hot when several candidate rows are evaluated together for that dimension.
- Per-dimension metadata can choose arithmetic instead of lookup for dimensions whose tables would be too large.
- It naturally supports a hybrid policy, such as lookup for $M_d \leq 31$ and direct arithmetic for larger M_d.

Cons of dimension-wise sign-separated tables.

- Node-at-a-time HNSW evaluation still accesses a different logical table for every dimension.
- Every contribution requires loading an offset or table pointer before loading the FP32 result.
- Variable table lengths make addresses less predictable for hardware prefetchers.
- SIMD lanes may target different tables and magnitudes, turning the operation into a high-latency gather.
- Thousands of independent allocations would be expensive and fragmented unless contiguous pools are used.
- Separate tables duplicate the magnitude-zero entry for both signs.
- Per-query construction cost remains proportional to $\sum_d (M_d + 1)$.
- Concurrent queries each require their own values because $r_{q,d}$ changes with the query.
- Dimensions with 16-bit-scale magnitudes can still create very large individual tables.

Dimension-major candidate batching. These tables become most effective when distance evaluation is reorganized:

```
for each dimension d:
    keep positive_table[d] and negative_table[d] hot
    for each candidate node c in a small batch:
        z = candidate[c].Z[d]
        distance[c] += table_for_sign(z)[abs(z)]
```

One dimension's pair of tables is then reused by all candidates before moving to dimension $d+1$. This resembles the current CUDA row-block kernel. However, it conflicts with stock hnswlib's scalar `DISTFUNC(left, right)` interface. Exploiting it during graph search would require batching the current node's unvisited neighbors or introducing a multi-candidate distance callback.

One signed table versus two sign tables. Another representation stores:

$$T_d[-M_d \dots M_d]$$

and addresses it with an offset such as `z + M_d`. It avoids choosing a positive or negative pointer and stores one zero entry, while the two-table design duplicates zero. The total entry

counts, $2M_d + 1$ versus $2(M_d + 1)$, are nearly identical. The signed table is often simpler for scalar indexing; separate sign tables can be convenient when the encoded node already provides sign and magnitude separately.

Recommendation for multiple tables. Store per-dimension maxima during database preparation, but do not unconditionally allocate every possible lookup entry. Use contiguous value pools plus offset arrays and choose a threshold M_{lookup}:

```
if M_d <= M_lookup:
    use positive/negative lookup tables for dimension d
else:
    use direct SIMD arithmetic for dimension d
```

For the unchanged stock HNSW callback, the direct mixed-type SIMD or precomputed-norm dot-product method remains the safest baseline. Multiple dimension-wise tables are most promising together with batched neighbor evaluation, small M_d, and enough distance calculations per query to amortize table creation.

21.16.4 Why the Access Pattern Can Still Be Slow

For one node, dimension d selects a different table row and the stored magnitude selects a position inside that row:

```
sum += table[dimension][sign][magnitude];
```

As dimensions advance, the base address jumps by one complete magnitude table, while magnitudes vary according to the node. A single node distance therefore tends to touch one cache line in each of many table rows. With $D = 3584$, table loads can generate substantially more cache traffic than the small integer node itself.

The existing CUDA code avoids a full $D \times M$ table. It creates small positive and negative magnitude tables for one dimension at a time in shared memory and lets 32 row lanes reuse them. That pattern is effective because many database rows are processed together for the same dimension. Stock HNSW requests distances for irregular candidate nodes one at a time, so it cannot obtain the same reuse without batching neighbor-distance evaluation.

21.16.5 Pros of the Lookup-Table Design

- The lookup value is exact for a full-precision query residual.
- It removes per-candidate integer-to-float conversion, multiplication by Δ, subtraction, and squaring.
- The table is reused for every HNSW distance calculation made by one query.
- It is attractive when magnitudes are very small and tightly bounded.
- It becomes more attractive if several candidate rows are processed dimension-major as a batch, because they reuse one hot table row.
- Zero bins require only the magnitude-zero entry.

21.16.6 Cons of the Lookup-Table Design

- Memory grows with $D(M_{\max}+1)$, not merely with D.
- A full table for a 16-bit magnitude range is prohibitively large.
- One table per concurrent query can pressure the last-level cache and memory allocator.
- Indexed loads and SIMD gathers can be slower than regular vector arithmetic when table entries miss cache.
- Table construction adds $O(DM_{\max})$ query-preparation work.
- Stock HNSW's one-candidate-at-a-time callback offers little cross-candidate table reuse.
- Per-dimension maximum magnitudes can reduce memory, but variable row offsets make addressing and SIMD gathering more complicated.
- The table accelerates only the residual squared term; alpha, gamma, graph traversal, and pointer chasing remain.

21.16.7 Lower-Memory Algebraic Alternative

The residual term can be expanded without a large table:

$$\|r_q - \Delta z_i\|^2 = \|r_q\|^2 \Delta^2 \|z_i\|^2 - 2\Delta r_q^T z_i.$$

For each query, compute $\|r_q\|^2$ once. For each database node, precompute the integer norm:

$$n_i = \|z_i\|^2.$$

Every query-to-node residual distance then needs only a mixed FP32/signed-bin dot product:

$$\|r_q\|^2 + \Delta^2 n_i - 2\Delta r_q^T z_i.$$

This uses regular contiguous SIMD loads, has $O(D)$ query memory, and avoids the per-query $O(DM_{\max})$ table build. It adds one stored norm per node and still requires widening bins to FP32 for the dot product. On modern CPUs, the predictable memory access and vector multiply-add operations may be faster than cache-missing table gathers.

21.16.8 Tiled and Hybrid Lookup Alternatives

If profiling shows that arithmetic conversion is a bottleneck, smaller lookup designs can be tested:

- build tables only when $M_{\max}$ is below a configured threshold;
- use direct SIMD arithmetic for 16-bit nodes and lookup tables only for 8-bit nodes;
- allocate tables from actual per-dimension maxima rather than the global maximum;
- build a table for a small dimension tile, process a batch of candidates, and reuse the buffer for the next tile;
- cache tables lazily for frequently reached dimensions or blocks; or

- change HNSW neighbor evaluation to batch several candidate ids so one dimension's table remains hot.

The tiled candidate-batch design resembles the existing GPU kernel most closely, but it requires a broader change than a custom scalar `DISTFUNC` because HNSW currently asks that callback for one pair at a time.

21.16.9 Recommendation

Implement the direct mixed-type SIMD formula first, preferably using the precomputed node norm and dot-product identity. It has bounded per-query memory, regular accesses, and works for both 8-bit and 16-bit nodes.

Add the lookup-table implementation as an optional runtime strategy for small-magnitude 8-bit data. Select it only after measuring:

- table construction time;
- HNSW distance-computation count per query;
- L1, L2, and last-level cache miss rates;
- single-query latency and multithreaded QPS;
- table memory per concurrent query; and
- speed relative to direct AVX-512 conversion and FMA.

This keeps the full-precision-query guarantee while allowing the lookup idea to be used where its saved arithmetic exceeds its cache cost.

21.17 Optional Cleaner API Change

The tagged-descriptor approach requires no modification to `hnswalg.h` and is the minimal first implementation. A cleaner future interface can give the space two explicit callbacks:

```
node_to_node_distance(node_a, node_b, context)
query_to_node_distance(query, node, context)
```

Construction would call the first callback and search would call the second. This removes runtime tag checking and makes it impossible to copy a query descriptor as a stored node, but it touches every relevant HNSW distance-call site. It should be considered after the custom-space prototype validates the formulas and performance.

21.18 Compatibility with the hnswlib Distance Interface

This node format cannot use `L2Space` or `L2SpaceI`. `L2Space` expects float rows. `L2SpaceI` expects unsigned bytes, ignores alpha and beta, returns an integer distance, and does not represent signed residual bins.

A custom `SpaceInterface<float>` must:

- report the fixed byte size of one node payload;
- provide a node-to-node corrected distance for graph construction;

- provide query-to-node distance during search;
- hold a non-owning pointer to immutable global model context; and
- use thread-local query state or an explicit query descriptor.

Stock hnswlib uses one generic distance callback for construction and search. A tagged first field can distinguish a stored node descriptor from a temporary query descriptor, but the safer design is to add separate node-to-node and query-to-node callbacks. A query descriptor may point to temporary a_q, r_q, and γ_q buffers; those buffers must remain alive until the synchronous `searchKnn` call returns.

21.19 Proposed Data-Preparation Algorithm

```
INPUT:
    compressed package, rows N, dimensions D, rank R, error bound eb

1. Parse and validate package metadata.
2. Set Delta = 2 * eb.
3. Scan every nonzero residual header and decoded magnitude:
       max_magnitude = max(max_magnitude, magnitude)
4. Select one global signed code type:
       int8 if max_magnitude <= 127
       int16 if max_magnitude <= 32767
       otherwise int32 or fail/recompress
5. Allocate row-major Z[N][D] in the selected type and initialize to zero.
6. For dimension d = 0 .. D-1:
       For each 32-row block:
           if zero-block mask bit is zero:
               continue
           read magnitude bit width from header
           for each valid lane:
               decode sign and magnitude
               Z[row_start + lane][d] = signed magnitude
7. Dequantize the shared basis and row-wise alpha.
8. Recommended exact path:
       compute thin QR: B = Q*T
       for each row i:
           a[i] = T * alpha[i]
           gamma[i] = Q^T * (Delta * Z[i])
9. Build one fixed-size NodePayload per row.
10. Preserve original row id as the public HNSW label.
OUTPUT:
    immutable global context and fixed-size row-major node payloads
```

Steps 6 and 8 can be fused or tiled to avoid rereading Z, but a simple correct implementation should be validated before applying that optimization. If the existing CUDA beta preparation is reused without QR, Step 8 instead keeps α_i and $\beta_i = B^T(\Delta z_i)$, and the graph uses either the current approximate coefficient norm or the global Gram matrix.

21.20 Proposed HNSW Construction Algorithm

```
Create CompressedResidualSpace(D, R, Delta, code_width, global_context)
Create HierarchicalNSW<float>(space, N, M, ef_construction)

For each row i:
    node.row_id = original_row_id[i]
    node.coefficient = a[i] // or alpha[i]
    node.residual_beta = gamma[i] // or beta[i]
    node.residual_bin = Z[i]
    index.addPoint(&node, node.row_id)

NodeDistance(i, j):
    coefficient = squared_L2(a_i, a_j)
    residual_integer = sum_d int64(Z_i[d] - Z_j[d])^2
    residual = Delta^2 * residual_integer
    cross = 2 * dot(a_i - a_j, gamma_i - gamma_j)
    return clamp_small_negative(coefficient + residual + cross)
```

The accumulation must widen before subtraction and multiplication. For 16-bit codes, a difference can be as large as 65534, whose square does not fit in a signed 32-bit integer. An `int64_t` sum is required, and the implementation must cast to a sufficiently wide type before squaring.

21.21 Proposed HNSW Query Algorithm

```
PrepareQuery(q):
    q_centered = q - mean
    a_q = transpose(Q) * q_centered
    r_q = q_centered - Q * a_q
    gamma_q = transpose(Q) * r_q
    return QueryDescriptor(a_q, r_q, gamma_q)

QueryToNodeDistance(query, node_i):
    coefficient = squared_L2(query.a, node_i.a)
    residual = sum_d (query.r[d] - Delta * node_i.Z[d])^2
    cross = 2 * dot(
        query.a - node_i.a,
        query.gamma - node_i.gamma)
    return clamp_small_negative(coefficient + residual + cross)

Search:
    descriptor = PrepareQuery(q)
    candidates = index.searchKnn(descriptor, k)
    return candidates
```

If the graph distance exactly matches reconstructed squared L2, ordinary HNSW search can return its final candidates directly. An optional exact float-reconstruction rerank can still be used as a validation tool. If the current approximate $B^T B \approx I$ formula or quantized query residual is used, retrieving more than k candidates and reranking is safer.

21.22 SIMD Implementation Opportunities

The row-major fixed-width layout is much more SIMD-friendly than the original variable-bit stream.

Integer residual term. AVX2 or AVX-512BW can load contiguous signed 8-bit or 16-bit bins from two nodes. Values must be widened before subtraction to avoid wraparound. The differences are squared and accumulated into sufficiently wide lanes. The integer sum is converted to float or double once and multiplied by $\Delta^2 = 4\epsilon^2$.

Query residual term. SIMD loads signed bins, widens them, converts them to float, multiplies by Δ, subtracts from float query residual lanes, and uses fused multiply-add for squared accumulation.

Alpha and beta terms. Coefficient and beta vectors are contiguous rank-dimensional floats. AVX-512 can calculate coefficient L2 and the cross-term dot product in the same pass. Padding rank to a multiple of 16 with zeros can eliminate scalar tails.

Preparation. The original dimension-major organization is efficient for decoding 32 lanes of one dimension. Preparation should decode a complete block in parallel and scatter or transpose those 32 values into row-major output. GPU preparation can follow the current one-warp-per-row-block pattern; CPU preparation can use blocked transposition to avoid cache-unfriendly single-value writes.

The custom distance function, rather than `space_l2.h`, must own these SIMD kernels because it combines integer and float terms.

21.23 Memory Consequences for D=3584 and R=64

Ignoring alignment and metadata, a normal float vector requires:

$$3584 \cdot 4 = 14336 \text{ bytes/node.}$$

With signed 8-bit residual bins plus float coefficient and beta:

$$3584 + 2(64 \cdot 4) = 4096 \text{ bytes/node.}$$

This is approximately 3.5 times smaller than the float vector payload. With signed 16-bit bins:

$$3584 \cdot 2 + 2(64 \cdot 4) = 7680 \text{ bytes/node,}$$

which is approximately 1.87 times smaller. Graph links, labels, deletion state, alignment, and allocator overhead are additional in every case.

This representation is larger than the original bit-packed residual stream because zero-block compression and per-block variable bit widths are lost. Its benefit is a fixed-size, row-contiguous, random-access node format with much cheaper distance computation.

21.24 KMeans and Cluster-Centered Models

The simple derivation above is directly valid when all rows share the same mean and global basis. KMeans requires identifying which model is used.

Common global basis with cluster centroids. If:

$$\widehat{X}_i = c_{g_i} + B\alpha_i + \Delta z_i,$$

the centroid does not cancel for nodes in different clusters. With QR $B = QT$, define:

$$u_i = c_{g_i} + \Delta z_i, \qquad \gamma_i = Q^T u_i.$$

Then:

$$d(i,j) = \|a_i - a_j\|^2 + \|u_i - u_j\|^2 + 2(a_i - a_j)^T(\gamma_i - \gamma_j).$$

The D-dimensional loop computes $\|c_{g_i} - c_{g_j} + \Delta(z_i - z_j)\|^2$ using globally stored centroids and the two cluster ids. This remains $O(D + R)$ and is exact for the reconstructed representation.

Different basis per cluster. If each cluster has its own B_g, two nodes in different clusters do not share one orthonormal coordinate system. Row-wise alpha and beta under each node's own basis are insufficient for the simple formula. Exact cross-cluster distance requires cross-basis matrices, more projections, or direct dimension-wise reconstruction of the low-rank difference.

A cleaner architecture is one HNSW per cluster plus a centroid-level routing index, followed by merging candidates from several clusters. This preserves the simple within-cluster formula but introduces routing recall as another parameter. Merely storing cluster assignments globally does not by itself make the global corrected formula valid.

21.25 Feasibility Verdict

Is the proposal possible? Yes. For mean-centered global rSVD, it can produce an HNSW graph over the reconstructed lossy vectors without storing full float rows. With QR reparameterization, row-wise signed bins, and consistent beta values, the node-to-node distance can be exact squared L2 for that reconstructed dataset.

What is gained?

- No $N \times D$ reconstructed float matrix is stored.
- Node residuals are fixed-size and support direct random access.
- Database construction distance uses integer residual arithmetic.
- Row-major codes allow AVX2/AVX-512 processing.
- Precomputed beta reduces exact low-rank/residual interaction from an $O(DR)$ reconstruction to an $O(R)$ cross term.
- HNSW graph topology can use the reconstructed-data distance rather than an alpha-only surrogate.

What is lost or made more expensive?

- Every residual bin is unpacked and retained, so the compact zero-block and variable-bit representation is expanded.
- Node payload remains $O(D)$, although with one or two bytes rather than four bytes per residual coordinate.
- Data preparation requires a full dimension-major to row-major transpose.
- Beta adds $N \times R$ floats, and transformed alpha adds another $N \times R$ floats.
- A custom hnswlib space, query descriptor, SIMD kernel, and index serialization contract are required.
- Cluster-wise bases require a more complex distance or multiple cluster indexes.
- The graph must be tied to the exact package, row mapping, code width, error bound, and transform version.

21.26 Required Correctness Checks Before Code Generation

The implementation should not proceed until the following invariants are captured in tests:

1. Decoded signed $Z[i, d]$ exactly matches the existing residual decoder for zero and nonzero blocks.
2. $m_{\max}$ fits the selected signed node type.
3. $\Delta Z[i, d]$ matches the residual used by the current decompressor.
4. Stored beta or gamma matches a reference matrix-vector projection.
5. The corrected node-to-node formula matches ordinary squared L2 between two explicitly reconstructed rows within tolerance.
6. The query-to-node formula matches squared L2 between the raw query and the explicitly reconstructed row.
7. SIMD and scalar custom distances agree.
8. Single-threaded and multithreaded graph builds use only immutable global context and thread-local query/scratch state.
9. Saved and loaded indexes verify package hash, D, R, Δ, code width, QR/Gram mode, and row mapping.
10. For KMeans, original row ids, reordered row ids, and cluster ids remain consistent through insertion and result reporting.

21.27 Recommended Design Decision

For the first implementation, use mean-centered global rSVD, one globally selected signed code width, row-major residual bins, and the QR-based exact formula. Store transformed alpha and gamma contiguously with each node unless memory profiling strongly favors external arrays. Keep the original bit-packed package separately as the durable compressed source, and treat the HNSW file as a derived index.

This first version should implement scalar distance and exhaustive formula-validation tests before adding AVX-512. Once exact agreement is established, add SIMD for integer residual accumulation and rank terms, then measure graph build time, QPS, recall, peak memory, and index size against:

- ordinary float HNSW over reconstructed rows;
- alpha-only HNSW with exact reranking; and
- the original compressed-direct CUDA search.

That comparison will show whether the larger row-wise code index earns enough construction and query throughput to justify expanding the original bit-packed residual representation.

22 Implemented CPU CPRQ HNSW Construction and Full-Precision Search

This section records the implemented experimental path for constructing and querying HNSW directly from the mean-centered global-rSVD CPRQ package. The implementation is deliberately isolated from the original hnswlib source. No file under `hnswlib/hnswlib` is modified.

The new implementation is located in:

- `hnsw_qlrpcomp/cprq_package_cpu.h`;
- `hnsw_qlrpcomp/space_cprq_l2.h`;
- `hnsw_qlrpcomp/example_search_cprq_compressed.cpp`;
- `hnsw_qlrpcomp/export_nvcomp_package_raw.cpp`; and
- `hnsw_qlrpcomp/CMakeLists.txt`.

The existing public classes `hnswlib::SpaceInterface<float>` and `hnswlib::HierarchicalNSW<float>` are reused. The changed storage and distance strategy is supplied entirely through a new space class and its fixed-size node payload.

22.1 Package Input and Outer Zstd Handling

The requested package is:

```
/tmp/qlrpcomp_gpu_pool_mean_rSVD_model_components_r64_qb_8_abs0905/
  embeds_0[(16,20,36,3584)]_w_encoding_full_package/
  w_encoding_full_package_zstd.bin
```

At the time of implementation, the corresponding `w_encoding_full_package_raw.bin` was not present. The available file is not one ordinary Zstd frame. It is an nvCOMP-native container containing a chunk table followed by standard Zstd-compressed chunks.

Two paths are provided:

1. `export_nvcomp_package_raw.cpp` calls the existing project method `cprq_gpu_pool_flow::cprq_nvcomp_zstd_de` and writes the raw package. This conversion requires a CUDA device because that existing method uses nvCOMP on the GPU.
2. The example's CPU loader can also parse the nvCOMP-native chunk table and decompress each embedded Zstd frame through `libzstd`. This allowed CPU testing when the execution sandbox did not expose a CUDA device.

Once the outer lossless layer is removed, all package parsing, residual-bin decoding, QR preparation, beta preparation, HNSW construction, and HNSW search are CPU operations.

22.2 Raw Full-Package Parsing

The raw package parser consumes native-endian fields in exactly the order written by `cprq_build_w_encoding_full_p` :

1. magic $0x51505745$ and version 1;

2. rows, columns, manual block size, and absolute error bound;

3. rank, oversampling, maximum magnitude, blocks per dimension, and residual block size;

4. encoded residual-stream size;

5. quantized mean block;

6. quantized basis block;

7. quantized alpha block; and

8. the encoded residual byte vector.

Each quantized model block contains minimum, maximum, rows, columns, bit width, a 64-bit value count, and the quantized bytes. For b bits, the CPU dequantizer uses:

$$s = \frac{x_{\max} - x_{\min}}{2^b - 1}, \qquad x_j = x_{\min} + s\, q_j.$$

The loader verifies all expected shapes:

$$\mu : D, \qquad B : D \times R, \qquad \alpha : N \times R.$$

22.3 Actual Block-Mask Residual Layout

The package used for the experiment was generated by the block-mask encoder, not by the older repeated `[header][sign][magnitude]` layout. Its residual byte vector is:

$$\underbrace{\text{mask words}}_{\lceil NB/32 \rceil \text{ uint32 values}} \;\|\; \underbrace{\text{nonzero headers}}_{\text{one byte per nonzero block}} \;\|\; \underbrace{\text{nonzero payloads}}_{\text{sign bytes and magnitude bytes}}.$$

Here:

$$NB = D \left\lceil \frac{N}{S} \right\rceil,$$

where S is the residual block size. Logical blocks remain dimension-major:

$$\text{blockId} = d \cdot \text{blocksPerDimension} + \text{blockWithinDimension}.$$

A zero mask bit means that all bins in the block are zero and that the block has neither a header nor a payload. A one mask bit selects the next compact header and payload.

For a nonzero block containing n_b rows and magnitude width w:

$$\text{signBytes} = \left\lceil \frac{n_b}{8} \right\rceil,$$

$$\text{magnitudeBytes} = w \left\lceil \frac{n_b}{8} \right\rceil.$$

The current block-mask encoder packs signs least-significant-bit first. It packs magnitude values as a row-major bitstream: the bit for local row ℓ and magnitude bit b is at:

$$p = \ell w + b.$$

Its byte and bit positions are $p/8$ and $p \bmod 8$. This detail differs from the older bitplane-major decoder and must not be interchanged with it.

22.4 Signed Int8 Conversion and Clipping

The decoder writes a row-major matrix:

$$Z_8 \in \{-128, \ldots, 127\}^{N \times D}.$$

For every decoded bin z:

$$z_8 = \min(127, \max(-128, z)).$$

The residual represented by the node is:

$$e_{8,i} = \Delta z_{8,i}, \qquad \Delta = 2\epsilon.$$

For every row, the loader also calculates:

$$n_i = \sum_{d=0}^{D-1} z_{8,i,d}^2.$$

The tested package reported a stored and decoded maximum magnitude of 42. Consequently, all values fit in signed 8-bit storage and both clipping counts were zero. If clipping occurs with another package, beta and the squared bin norm are calculated from the clipped bins, so the node remains internally consistent even though the original error-bound guarantee no longer holds.

22.5 QR Transformation of Alpha and Beta

The dequantized basis can lose exact orthogonality through model quantization. Directly assuming $B^T B = I$ would therefore introduce a distance error. Calculating:

$$(\alpha_i - \alpha_j)^T (B^T B)(\alpha_i - \alpha_j)$$

for every HNSW comparison would instead require $O(R^2)$ work.
The CPU preparation performs a thin QR factorization:

$$B = QT, \qquad Q^T Q = I,$$

where Q is $D \times R$ and T is upper triangular. It then prepares the following node-wise values:

$$a_i = T\alpha_i,$$

$$\gamma_i = Q^T(\Delta z_{8,i}).$$

In the code, γ_i is named `transformed_beta` because it is the beta correction in the orthonormal coordinate system. This differs from the untransformed CUDA quantity $B^T(\Delta z_i)$, but it carries the same cross-term role while allowing an $O(R)$ distance.

The beta preparation is the CPU equivalent of the logical operation in `cprq_prepare_database_residual_beta_me` : for every row, accumulate the decoded residual times every basis component. The implementation traverses each row's contiguous signed bins and updates 16 rank components at a time with AVX-512, or eight with AVX2.

22.6 Fixed HNSW Node Payload

For $D = 3584$ and $R = 64$, every level-zero HNSW node stores this fixed payload:

Field	Type	Bytes
Node tag and reserved word	two `uint32_t`	8
Transformed alpha a_i	64 FP32 values	256
Transformed beta γ_i	64 FP32 values	256
Squared bin norm n_i	`uint32_t`	4
Signed residual bins $z_{8,i}$	3584 `int8_t` values	3584
Total		4108

The HNSW implementation copies these 4108 bytes into its own level-zero record in `addPoint`. The global mean, Q, Δ, rank, and dimension are owned by the custom space and are not duplicated per node.

22.7 Exact Node-to-Node Construction Distance

The stored reconstructed centered vector is:

$$\widehat{x}_{c,i} = Qa_i + \Delta z_{8,i}.$$

For nodes i and j, define:

$$\delta a = a_i - a_j.$$

Using $\gamma_i = Q^T(\Delta z_{8,i})$, the construction distance is:

$$d(i,j) = \|\delta a\|^2 + \Delta^2 \|z_{8,i} - z_{8,j}\|^2 + 2\delta a^T(\gamma_i - \gamma_j).$$

The signed bytes are widened before subtraction. This is required because the difference of two signed 8-bit values can be in $[-255, 255]$. AVX-512 loads 16 signed bytes, sign-extends them to 16 signed 32-bit integers, subtracts, squares, and accumulates.

22.8 Final HNSW Construction Algorithm

```
input:
    CPRQ full package
    M
    ef_construction

raw = remove_outer_lossless_layer(package)
header, q_mean, q_basis, q_alpha, residual_stream = parse(raw)

mean = dequantize(q_mean)
B = dequantize(q_basis)
alpha = dequantize(q_alpha)

Z8, norm = decode_block_mask_residuals_to_row_major_int8(
    residual_stream)

Q, T = thin_qr(B)
```

```
parallel for row i:
    transformed_alpha[i] = T * alpha[i]
    transformed_beta[i] = transpose(Q) * (Delta * Z8[i])

space = CprqL2Space(mean, Q, Delta, D, R)
index = HierarchicalNSW(space, N, M, ef_construction)

for row i in [0, N):
    node.tag = NODE
    node.alpha = transformed_alpha[i]
    node.beta = transformed_beta[i]
    node.norm = sum(Z8[i] * Z8[i])
    node.bins = Z8[i]
    index.addPoint(node, label=i)
```

During graph construction, every distance callback receives two node-tagged payloads and executes the node-to-node formula above. All layer generation, candidate expansion, heuristic neighbor selection, reciprocal edge updates, and locking remain the original hnswlib behavior.

22.8.1 Summary: Why the CPU Applies QR but the GPU Does Not

For the centered database matrix, the rSVD model has the conventional form:

$$X_c \approx U\Sigma V^T.$$

The row-wise coefficient matrix is $\alpha = U\Sigma$. The package stores the global basis as a $D \times R$ matrix $B = V$, so an individual centered row is reconstructed as:

$$\widehat{x}_{c,i} = B\alpha_i + \Delta z_i.$$

Before model quantization, the rSVD basis is theoretically orthonormal:

$$V^T V = I.$$

The GPU can therefore use the low-rank basis directly as its projection basis. More importantly, the exact GPU distance kernel does not require an orthogonality assumption: it explicitly evaluates the low-rank difference through the stored basis for every dimension,

$$d(q,i) = \sum_{d=0}^{D-1} \left[B_d(\alpha_q - \alpha_i) + r_{q,d} - \Delta z_{i,d}\right]^2.$$

This dimension-wise expression remains valid even if quantizing and dequantizing V produces a basis $\widehat{V}$ for which $\widehat{V}^T\widehat{V} \neq I$. Consequently, the exact GPU path does not need to factor the basis again. A GPU fast path that replaces the explicit basis multiplication with $\|\alpha_q - \alpha_i\|^2$, however, relies on orthogonality or accepts an approximation.

HNSW evaluates a very large number of node-to-node distances during graph construction. Repeating the GPU-style $D \times R$ basis multiplication for each comparison would be too expensive. The CPU implementation therefore factors the already dequantized basis once:

$$\widehat{V} = QT, \qquad Q^T Q = I,$$

and transforms every node coefficient:

$$a_i = T\alpha_i.$$

This does not recompute rSVD, change the rank, or modify the package. It preserves the reconstructed low-rank vector because:

$$\widehat{V}\alpha_i = Q(T\alpha_i) = Qa_i.$$

The orthonormal representation permits the exact $O(D + R)$ HNSW formulas using node-wise beta values $\gamma_i = Q^T(\Delta z_i)$. It also makes the full-precision query residual orthogonal to Q, so query beta is approximately zero.

QR is not the only valid design. Without QR, the implementation would need the global Gram matrix:

$$G = \widehat{V}^T\widehat{V},$$

and the low-rank term would become:

$$(\alpha_i - \alpha_j)^T G(\alpha_i - \alpha_j).$$

That alternative also requires consistent database and query cross terms and normally introduces $O(R^2)$ work per comparison. The one-time QR transformation was selected to retain the GPU exact-distance meaning while making repeated CPU graph construction and search practical.

22.9 Full-Precision Query Preparation

The external query remains FP32. It is centered:

$$q_c = q - \mu.$$

It is projected into the orthonormal low-rank space:

$$a_q = Q^T q_c.$$

The full-precision query residual is:

$$r_q = q_c - Qa_q.$$

The query preparation also stores:

$$\rho_q = \|r_q\|^2.$$

Because a_q is an orthogonal projection:

$$Q^T r_q \approx 0.$$

Therefore, an explicit query beta array is unnecessary. Small floating-point projection error is absorbed by normal FP32 numerical error.

The query is represented by a temporary tagged descriptor containing pointers to a_q and r_q, plus ρ_q. It is not inserted into HNSW and is valid only for the duration of `searchKnn`.

22.10 Exact Query-to-Node Distance Without a Cache Table

For node i:

$$d(q,i) = \|a_q - a_i\|^2 + \|r_q - \Delta z_{8,i}\|^2 - 2(a_q - a_i)^T \gamma_i.$$

The residual term is expanded as:

$$\|r_q - \Delta z_{8,i}\|^2 = \rho_q + \Delta^2 n_i - 2\Delta(r_q^T z_{8,i}).$$

Thus the implemented callback is:

$$d(q,i) = \|a_q - a_i\|^2 + \rho_q + \Delta^2 n_i - 2\Delta(r_q^T z_{8,i}) - 2(a_q - a_i)^T \gamma_i.$$

No query lookup table and no D-dimensional FP32 database reconstruction are required. For the mixed dot product $r_q^T z_{8,i}$, AVX-512:

1. loads 16 signed residual bytes;
2. sign-extends them to 16 signed 32-bit integers;
3. converts the integers to FP32;
4. loads 16 FP32 query residuals; and
5. performs fused multiply-add accumulation.

22.11 Final SearchKNN Algorithm

```
input:
    full_precision_query q
    requested neighbor count k
    ef_search

q_centered = q - global_mean
query_alpha = transpose(Q) * q_centered
query_residual = q_centered - Q * query_alpha
query_residual_norm = dot(query_residual, query_residual)

query_view.tag = QUERY
query_view.alpha = query_alpha
query_view.residual = query_residual
query_view.residual_norm = query_residual_norm

index.setEf(ef_search)
result = index.searchKnn(&query_view, k)
```

Inside the original HNSW traversal:

1. the upper-layer greedy search invokes the custom callback with the query descriptor and a stored node;
2. level-zero best-first expansion uses the same callback;
3. visited-state, candidate heaps, result heaps, and graph-neighbor access remain unchanged; and
4. only the payload interpretation and distance arithmetic differ from ordinary `L2Space`.

22.12 Measured Validation Results

The tested package properties were:

- $N = 11520$, $D = 3584$, and $R = 64$;
- residual block size 32;
- $\epsilon = 0.0905$ and $\Delta = 0.181$;
- maximum decoded magnitude 42;
- zero clipped values; and
- 4108 payload bytes per HNSW node.

With 16 OpenMP preparation threads, $M = 16$, efConstruction = 200, and efSearch = 200, one run reported:

Measurement	Result
Package decode, QR, alpha, and beta preparation	1.1035 s
HNSW construction	7.8725 s
Construction throughput	1463.31 vectors/s
Queries	1000
HNSW-only search time	0.6880 s
HNSW-only QPS	1453.41
End-to-end query time	1.0044 s
End-to-end QPS, including query preparation	995.61

The AVX-512 algebra was checked against explicit FP64 reconstruction:

- query-to-node absolute distance difference: 2.1×10^{-5}; and
- node-to-node absolute distance difference: 2.31×10^{-4}.

These checks confirm that the construction and query formulas represent the stored reconstructed vectors to normal FP32 accuracy.

Interpretation of self-label recall. The requested query file is from a separately decompressed dataset path and is not the exact reconstruction stored by this mean-rSVD package. For example, row zero has squared distance approximately 604.96 from node zero's reconstruction. Therefore the measured self-label rate of 0.607 for 1000 queries is not a valid HNSW recall measurement against exact ground truth. Meaningful ANN recall requires either original external queries with an exact ground-truth neighbor file, or brute-force ground truth calculated against the same reconstructed CPRQ database represented by the HNSW nodes.

22.13 Build and Run Location

Step-by-step commands, argument order, raw-package export, and log redirection are recorded in:

```
hnsw_qlrpcomp/README.md
```

The main executable takes:

```
M ef_construction ef_search query_count
compressed_package query_fvecs [optional_index_output]
```

23 Summary

In this repository, HNSW is implemented as a CPU-resident, pointer-based, header-only C++ graph index. It stores every inserted vector inside the index memory, stores level-0 graph links beside the vector and label in a contiguous per-node record, stores upper-layer links in per-node heap allocations, and uses dense internal ids for graph edges. Public labels are maintained through an unordered map and are what the user sees in query results. Construction is controlled mainly by `M` and `ef_construction`, while query accuracy and speed are controlled mainly by `ef` and `k`.